Part A

Recidivist = R.

Step 1 R = true 3

R = false 3

$$H(R) = -\frac{3}{6}\log_2\frac{3}{6} \times 2 = 1$$

Step 2 • Good Behaviour (G) = false

R = true 2

R = false 1

$$H = -\frac{2}{3}\log\frac{2}{3} - \frac{1}{3}\log_2\frac{1}{3} \simeq 0.918$$

• G = true

R = true 1

R = false 2

$$H = -\frac{1}{3}\log\frac{1}{3} - \frac{2}{3}\log\frac{2}{3} = 0.918$$

$$\text{Remainder}(G) = \frac{3}{6}(0.918) \times 2 = 0.918$$

$$\text{Info gain} = 1 - 0.918 = 0.082$$

• Age < 30 (A) = true

R = true 2

R = false 0

H = 0

A = false

R = true 1

R = false 4

$$H = -\frac{1}{4}\log\frac{1}{4} - \frac{3}{4}\log\frac{3}{4} = 0.811$$

$$\text{Remainder}(A) = \frac{2}{6}\cdot 0 + \frac{4}{6}\,0.811 = 0.541$$

$$IG(A) = 0.459.$$

- Drug dependent (D) = true
  - R = true 1
  - R = false 0
  - H = 0

D = false
- R = true 2
- R = false 3
- H = 0.971
- IG = 0.191

=> A has max info gain → **root node**.

Subtrees

- Left branch (A = true)
  - All R = true => Leaf node : R = true.
- Right branch (A = false)
  - G = true
    - R = true 1
    - R = false 2
    - H = 0.918
    - IG = 0.122
  - G = false R = false

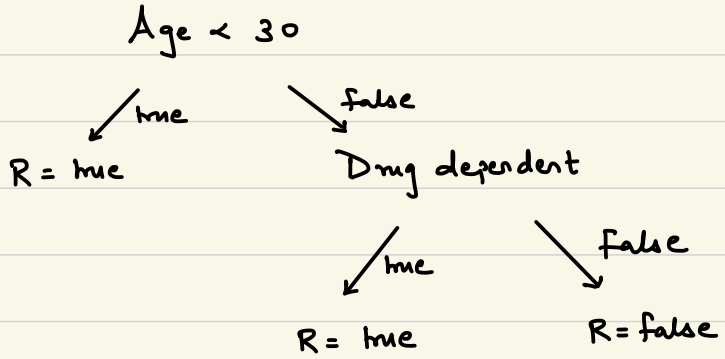
- D = true => R = true

  D = false => R = false

  IG = 0.811

  => choose D for this node

Final tree



Lab 4 - Tree-based Learning

In this lab, you will learn to:

- Build a decision tree manually
- Build a classifer using random forest with sklearn
- Visualize decision trees
- Build regression model using random forest
- Learn about additional performance metrics and hyper-parameter tuning.
- Build a classifier for particle physics data

Authors: Tuan Do & Bernie Boscoe

Latest Revision: Tuan Do

In [139…

```
# put your imports here
import numpy as np
import pandas as pd
import pylab as plt
```

Part A - Building a decision tree by hand

From Question 2 from Chapter 4 of Kelleher

A convicted criminal who reoffends after release is known as a recidivist. The table below lists a dataset that describes prisoners released on parole, and whether they reoffended within two years of release. This dataset lists six instances where prisoners were granted parole. Each of these instances are described in terms of three binary descriptive features. (GOOD BEHAVIOR, AGE ă 30, DRUG DEPENDENT) and a binary target feature, RECIDIVIST. The GOOD BEHAVIOR feature has a value of true if the prisoner had not committed any infringements during incarceration, the AGE ă 30 has a value of true if the prisoner was under 30 years of age when granted parole, and the DRUG DEPENDENT feature is true if the prisoner had a drug addiction at the time of parole. The target feature, RECIDIVIST, has a true value if the prisoner was arrested within two years of being released; otherwise it has a value of false.

ID	GOOD BEHAVIOR	AGE < 30	DRUG DEPENDENT	RECIDIVIST
1	false	true	false	true
2	false	false	false	false
3	false	true	false	true
4	true	false	false	false
5	true	false	true	true
6	true	false	false	false

Note: This example of predicting recidivism is based on a real application of machine learning: parole boards do rely on machine learning prediction models to help them when they are making their decisions. See Berk and Bleich (2013) for a recent comparison of different machine learning models used for this task. Datasets dealing with prisoner recidivism are available online, for exam- ple: catalog.data.gov/dataset/prisoner-recidivism/. The dataset presented here is not based on real data.

Part A1

Question 1

(5 pts)

Using this dataset, construct the decision tree that would be generated by the ID3 algorithm, using entropy-based information gain. Show your calculations for information gain and draw your final tree. Hint: follow the example in 4.3.1 in Kelleher. Note, you can upload a separate PDF for this question if you would like to draw or calculate on paper.

ANSWER : UPLOADED A SEPARATE PDF WITH THE TREE

Part A2

Question 2

(1 pt)

What prediction will the decision tree generated in part 1.1 return for the following query? GOOD BEHAVIOR = false,AGE < 30 = false, DRUG DEPENDENT = true

ANSWER : RECIDIVIST = TRUE

Part A3

Question 3

(1 pt)

What prediction will the decision tree generated in part 1.1 predict for the following query?

GOOD BEHAVIOR = true,AGE > 30 = true, DRUG DEPENDENT = false

ANSWER : RECIDIVIST = FALSE

Part A4

Question 4

(1 pt)

Discuss why this decision tree might lead to biases in inferences about recividism.

ANSWER **Biases in the Recidivism Decision Tree**

- The tree automatically predicts all parolees under 30 will reoffend, regardless of other factors. This creates systematic bias against younger individuals.
- GOOD BEHAVIOR is completely excluded from the model, meaning prisoners who demonstrated positive change during incarceration get no credit. This undermines rehabilitation efforts and fails to distinguish reformed individuals.
- DRUG DEPENDENT is treated as binary without considering treatment status, recovery progress, or addiction severity. Someone in active recovery is treated identically to someone with untreated addiction.
- With only 6 training instances, the tree likely captures noise rather than genuine patterns, leading to unreliable predictions when applied broadly.
- Age may correlate with protected characteristics like race or socioeconomic status, potentially perpetuating existing systemic inequalities in the criminal justice system.

These biases could result in unjust parole decisions and reinforce discriminatory patterns rather than fairly assessing individual risk.

Part B - Classify irises with a decision tree

Here, we'll repeat the iris classification problem with a decision tree using continuous features. We'll also be using the sklearn decision tree classifier.

In [140…
```
# load the data and the module to visualize decision trees
from sklearn.datasets import load_iris
from sklearn import tree
import numpy as np
import matplotlib.pyplot as plt
import sklearn as sk
from sklearn.metrics import accuracy_score
from sklearn import datasets, model_selection
import pandas as pd
import seaborn as sns
```

B1 Building a decision tree with 2 features

Question 5

(1 pt)

Load the iris data set. Let's start by using only **2 features: sepal length and width**

In [147…
```
# enter code here, hint: use your code from  your previous labs to load the
iris dataset
iris=datasets.load_iris()
data=iris.data[:,0:2] # This selects the two specified features -- sepal
lenght and sepal width
print("Selected features : ", iris.feature_names[0:2]) ## Verify that we
select the correct features
targets = iris.target
```

```
Selected features :  ['sepal length (cm)', 'sepal width (cm)']
```

Split the data into training and testing data

In [148…
```
# enter code here
x_train_all, x_validate, y_train_all, y_validate =
model_selection.train_test_split(data, targets, test_size=0.1)
x_train, x_test, y_train, y_test =
model_selection.train_test_split(x_train_all, y_train_all, test_size=0.2)
```

Question 6

(2 pts)

Load the classifier

See the sklearn documentation for more info: https://scikit-learn.org/stable/modules/generated/sklearn.tree.DecisionTreeClassifier.html

In [149…
```
# load the classifier
from sklearn.tree import DecisionTreeClassifier
```

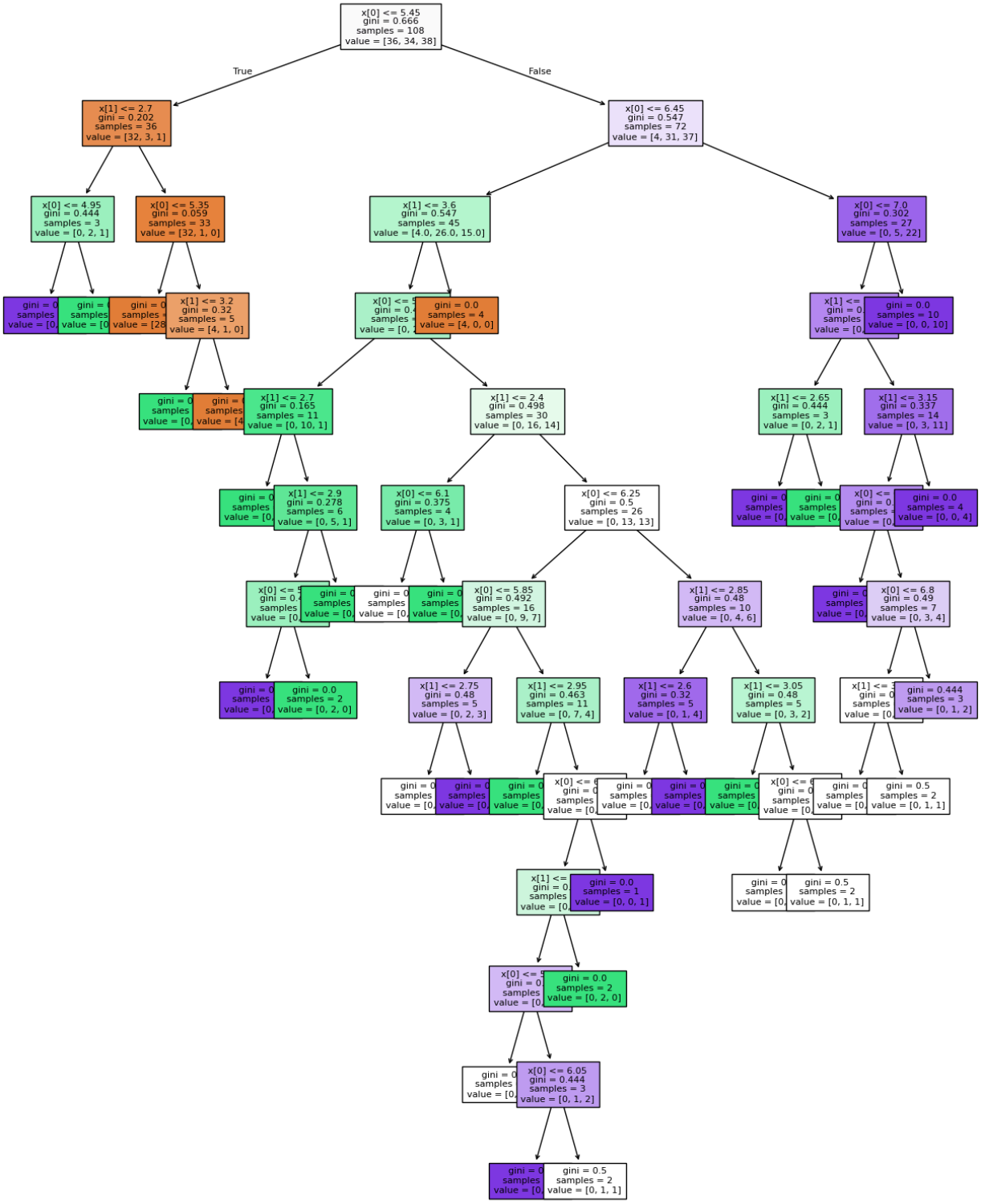
Train the classifier

In [150…
```
# enter your code here
clf=DecisionTreeClassifier()
clf.fit(x_train,y_train)
```

Out[150…
```
▾ DecisionTreeClassifier
DecisionTreeClassifier()
```

Visualize your decision tree. Sklearn has a nice function called tree.plot_tree that will plot the tree you created.

In [153…
```
# visualize the tree that was created
plt.figure(figsize=(15,20))
t = tree.plot_tree(clf,filled=True,fontsize=8)
```



Question 7

(2 pts)

Describe the tree that you created. **Where which feature did the root node use to split? Why? How deep is it? Why is it that deep?**

Description of the tree in the image above

1. ROOT NODE SPLIT : The feature used to split is the sepal length and the split value is sepal length less than or equal to 5.45
2. WHY THIS FEATURE ? : The decision tree algorithm chose sepal length at the root because it provides the best split to reduce the Gini impurity. At the root, we can see see that the impurity is 0.666 with all samples. By splitting on sepal length at 5.45, it separates the data into two groups that are more homogenous in terms of class distribution. This feature gave non trivial information gain compared to sepal width. This can be checked by looking at the impurity at the two leaves of the main root node, which are respectively 0.20 and 0.54.
3. TREE DEPTH : The tree is very deep 14.
4. WHY THIS DEEP ? : We put no depth constraint in our classifier, so the tree gorew until it reached pure leaf nodes. Additionally, we only used two features and hence the tree needed many more splits to spearate the iris classes. The tree is also trying to achieve perfect classification and this might very well lead to a risk of overfitting the data.

Test the classifier using the **accuracy score**.

In [154…
```
# enter your code here
y_pred = clf.predict(x_test)
print("accuracy:", accuracy_score(y_test, y_pred))
```

```
accuracy: 0.7037037037037037
```

Visualization: we can visualize how different pairs of iris features would give in terms of their classification in the iris problem. **Run the code cell below to see this decision surface for different pairs of features.**

In [156…
```
# Parameters
n_classes = 3
plot_colors = "ryb"
plot_step = 0.02

# Load data
iris = load_iris()
plt.figure(figsize=(12,12))

for pairidx, pair in enumerate([[0, 1], [0, 2], [0, 3],
                                [1, 2], [1, 3], [2, 3]]):
    # We only take the two corresponding features
    X = iris.data[:, pair]
    y = iris.target
```

```
    # Train
    clf = tree.DecisionTreeClassifier().fit(X, y)

    # Plot the decision boundary
    plt.subplot(2, 3, pairidx + 1)

    x_min, x_max = X[:, 0].min() - 1, X[:, 0].max() + 1
    y_min, y_max = X[:, 1].min() - 1, X[:, 1].max() + 1
    xx, yy = np.meshgrid(np.arange(x_min, x_max, plot_step),
                         np.arange(y_min, y_max, plot_step))
    plt.tight_layout(h_pad=0.5, w_pad=0.5, pad=2.5)

    Z = clf.predict(np.c_[xx.ravel(), yy.ravel()])
    Z = Z.reshape(xx.shape)
    cs = plt.contourf(xx, yy, Z, cmap=plt.cm.RdYlBu)

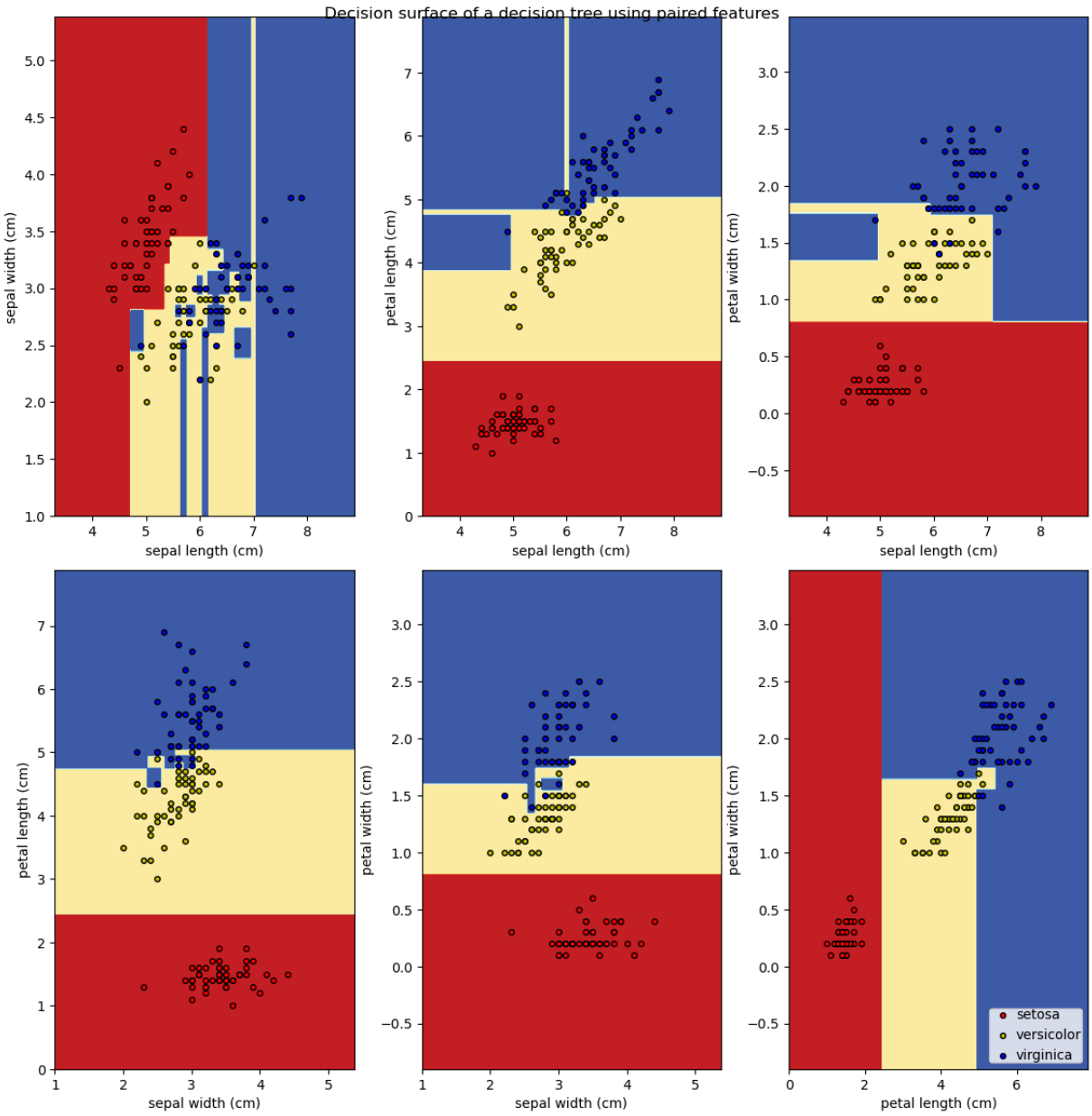
    plt.xlabel(iris.feature_names[pair[0]])
    plt.ylabel(iris.feature_names[pair[1]])

    # Plot the training points
    for i, color in zip(range(n_classes), plot_colors):
        idx = np.where(y == i)
        plt.scatter(X[idx, 0], X[idx, 1], c=color,
label=iris.target_names[i],
                    cmap=plt.cm.RdYlBu, edgecolor='black', s=15)

plt.suptitle("Decision surface of a decision tree using paired features")
plt.legend(loc='lower right', borderpad=0, handletextpad=0)
plt.axis("tight")
```

```
/var/folders/ww/jyhpvg8x3md06rt6f3f93jw00000gn/T/ipykernel_65216/1378001726.
py:38: UserWarning: No data for colormapping provided via 'c'. Parameters 'c
map' will be ignored
  plt.scatter(X[idx, 0], X[idx, 1], c=color, label=iris.target_names[i],
/var/folders/ww/jyhpvg8x3md06rt6f3f93jw00000gn/T/ipykernel_65216/1378001726.
py:38: UserWarning: No data for colormapping provided via 'c'. Parameters 'c
map' will be ignored
  plt.scatter(X[idx, 0], X[idx, 1], c=color, label=iris.target_names[i],
/var/folders/ww/jyhpvg8x3md06rt6f3f93jw00000gn/T/ipykernel_65216/1378001726.
py:38: UserWarning: No data for colormapping provided via 'c'. Parameters 'c
map' will be ignored
  plt.scatter(X[idx, 0], X[idx, 1], c=color, label=iris.target_names[i],
/var/folders/ww/jyhpvg8x3md06rt6f3f93jw00000gn/T/ipykernel_65216/1378001726.
py:38: UserWarning: No data for colormapping provided via 'c'. Parameters 'c
map' will be ignored
  plt.scatter(X[idx, 0], X[idx, 1], c=color, label=iris.target_names[i],
/var/folders/ww/jyhpvg8x3md06rt6f3f93jw00000gn/T/ipykernel_65216/1378001726.
py:38: UserWarning: No data for colormapping provided via 'c'. Parameters 'c
map' will be ignored
  plt.scatter(X[idx, 0], X[idx, 1], c=color, label=iris.target_names[i],
/var/folders/ww/jyhpvg8x3md06rt6f3f93jw00000gn/T/ipykernel_65216/1378001726.
py:38: UserWarning: No data for colormapping provided via 'c'. Parameters 'c
map' will be ignored
  plt.scatter(X[idx, 0], X[idx, 1], c=color, label=iris.target_names[i],
```

Out[156… (0.0, 7.88, -0.9, 3.480000000000004)



Question 8

(1 pt)

Based on the plot above, **identify some regions where overfitting is occurring.**

Enter your answer in this markdown cell. Two concrete instances of overfitting can be clearly seen,

1. In the graph for petal length vs sepal width -- we can see a very clear demarcation between the three iris species, with even some of the overlaps being fitted explcicitly.

2. In the graph for sepal width vs sepal length, the same is true !

Question 9

(2 pt)

By default, the decision tree can be too deep, which can make it overfit. **Recreate the classifier and use the keyword `max_depth` in the `DecisionTreeClassifier` object to specify a depth for your tree. Try refitting the data. Does your accuracy score improve?**

In [170…
```
# your code here
clf=DecisionTreeClassifier(max_depth=3) # A maximum depth of 3 nodes
clf.fit(x_train,y_train)
y_pred = clf.predict(x_test)
print("accuracy:", accuracy_score(y_test, y_pred))
```

```
accuracy: 0.6666666666666666
```

In [171…
```
# your code here
clf=DecisionTreeClassifier(max_depth=6) # A maximum depth of 6 nodes
clf.fit(x_train,y_train)
y_pred = clf.predict(x_test)
print("accuracy:", accuracy_score(y_test, y_pred))
```

```
accuracy: 0.8148148148148148
```

After setting the max_depth = 3, my test accuracy initially decreased. However, by experimenting with different depth values, I found that max_depth= $6 4 actually improved the accuracy. This shows that the orig inial inlimited tree of depth 14 was over fitting, while tree of depth 3 was too orestrictive. The optimal depth of 6$ provides the best balance between model complexity and generalization, demonstrating that regularization through depth limiting can improve performance when properly tuned.

B2 Building a decision tree with all features

Question 10

(4 pts)

Now that you have a sense of how the decision tree works with sklearn, **build a decision tree to classify irises using all the features and test it.**

In [ ]:
```
# your code here
data=iris.data
targets = iris.target
# split data
x_train_all_allfeatures, x_validate_allfeatures, y_train_all_allfeatures,
y_validate_allfeatures = model_selection.train_test_split(data, targets,
```

```
test_size=0.1)
x_train_allfeatures, x_test_allfeatures, y_train_allfeatures,
y_test_allfeatures =
model_selection.train_test_split(x_train_all_allfeatures,
y_train_all_allfeatures, test_size=0.2)
```

In [177…

```
#Load classifier
clf_allfeatures=DecisionTreeClassifier(max_depth=10)
# train classifier
clf_allfeatures.fit(x_train_allfeatures,y_train_allfeatures)
```

Out[177…

DecisionTreeClassifier

DecisionTreeClassifier(max_depth=10)

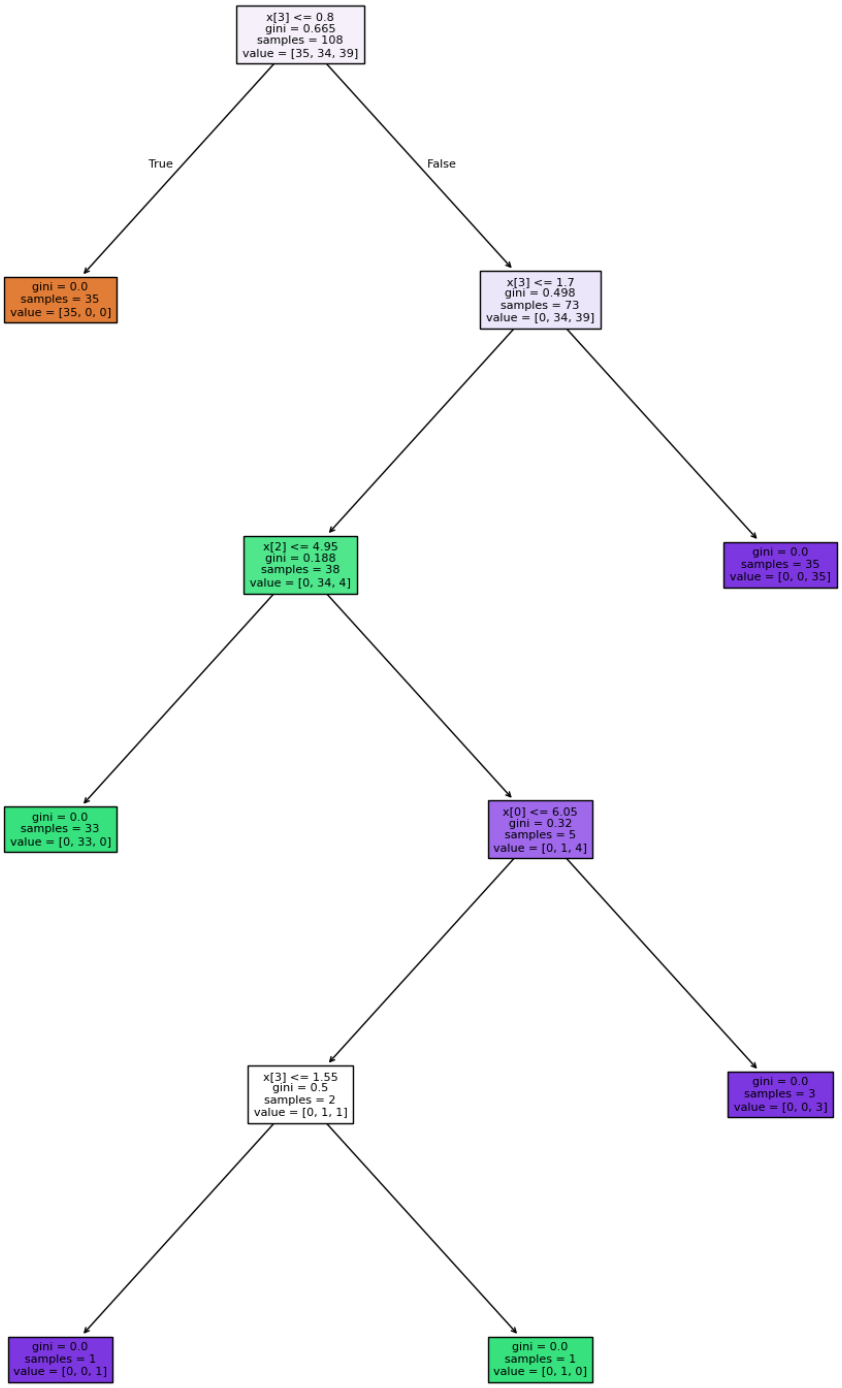
In [178…

```
# test all features classifier
y_pred_allfeatures = clf_allfeatures.predict(x_test_allfeatures)
print("accuracy:", accuracy_score(y_test_allfeatures, y_pred_allfeatures))
```

```
accuracy: 0.9629629629629629
```

In [180…

```
# visualize the tree that was created
plt.figure(figsize=(15,20))
t = tree.plot_tree(clf_allfeatures,filled=True,fontsize=8)
```



Question 11

(4 pts)

Based on your new tree, answer the following questions:

- **What feature does this tree split on? Why?**
- **What is your new accuracy score?**
- **Examine whether your score improves by changing `max_depth`.**
- By default, the classifer uses the Gini index to split the nodes. **What happens if you use information entropy instead?**

In [ ]:
```
# your answers here
```

1. In the tree above, the root node is the petal width and the split value is 0.8. The algorithms chose petal width because it provides the best information gain amongst all four features. We can clearly notice how effective this first split is, where the root has a gini of 0.665. The left node gets a gini of 0.0 so perfect purity. Hence petal width is much more discriminative than sepal length and width alone. The actual depth of the tree is 6.

In [182…
```
## I am changing the max_depth of the classifier and calculating the accuracy
for d in [1,2,3,5,10,25]:
    clf_allfeatures = DecisionTreeClassifier(max_depth=d, random_state=0)
    clf_allfeatures.fit(x_train_allfeatures, y_train_allfeatures)
    pred = clf_allfeatures.predict(x_test_allfeatures)
    print(d, accuracy_score(y_test_allfeatures, pred),
clf_allfeatures.get_depth(), clf_allfeatures.get_n_leaves())
```

```
1 0.6296296296296297 1 2
2 0.9629629629629629 2 3
3 0.9629629629629629 3 4
5 0.9629629629629629 5 6
10 0.9629629629629629 5 6
25 0.9629629629629629 5 6
```

2. The new accuracy score is 92.5%, which is a significant improvement from the 81% from when we considered just two features - sepal length and sepal width !!

3. Yes, the score increases as you move from max_depth 1 to 3, but after that it stays constant ! From the cell above this fact is clear !

4. Below I use information entropy rather than gini !

In [183…
```
# Use information entropy instead of gini -- keep the max_depth to be 3 ; all features considered
clf_allfeatures_entropy=DecisionTreeClassifier(max_depth=3,criterion="entropy")
# train classifier
clf_allfeatures_entropy.fit(x_train_allfeatures,y_train_allfeatures)
```

Out[183…

DecisionTreeClassifier

```
DecisionTreeClassifier(criterion='entropy', max_depth=3)
```

In [184…

```
# test all features classifier ; entropy instead of gini
y_pred_allfeatures_entropy =
clf_allfeatures_entropy.predict(x_test_allfeatures)
print("accuracy:", accuracy_score(y_test_allfeatures,
y_pred_allfeatures_entropy))
```

```
accuracy: 0.9629629629629629
```

In [187…

```
# Specifically for criterion = information entropy, for various values of
max_depth, calculate the accuracy of the decision tree classifier

for d in [1,2,3,5,10,25]:
    clf_allfeatures_entropy = DecisionTreeClassifier(max_depth=d,
random_state=0,criterion="entropy")
    clf_allfeatures_entropy.fit(x_train_allfeatures, y_train_allfeatures)
    pred_entropy = clf_allfeatures_entropy.predict(x_test_allfeatures)
    print(d, accuracy_score(y_test_allfeatures, pred_entropy),
clf_allfeatures_entropy.get_depth(),
clf_allfeatures_entropy.get_n_leaves())
```

```
1 0.6296296296296297 1 2
2 0.9629629629629629 2 3
3 0.9629629629629629 3 4
5 0.9629629629629629 5 6
10 0.9629629629629629 5 6
25 0.9629629629629629 5 6
```

We are observing that using entropy v/s Gini gives us essentially the same results. When using the entropy as the criterion instead of gini, I obtain the same accuracy score. This is expected because both metrics measure node impurity and tend to make similar splitting decisions. Both criteria aim to maximize information gain by creating the pures possible nides. The choice between them typically does not significantly impact the model performance !

Part C - Classify handwritten digits with random forest

As discussed in lecture and readings, usually we will not make just one decision tree, but rather use an emsemble of them called a random forest. Random forests have many applications and can even be used for image recognition. In this part, we will examine how we could use random forest to classifiy handwritten digits using another classic machine learning dataset.

This dataset is called MNIST and is used to test many machine learning models (for more information, see http://yann.lecun.com/exdb/mnist/). We'll use our usual framework for

an ML workflow for this part.

C1 Big Picture

Optical character recognition (OCR) is a very common machine learning task that is used to digitize handwriting. There is a lot of variation in how people write, but the number of possible characters is fairly small, so it is a well defined task. For this part of the lab, we'll try to identify numbers, which means there will be 10 classes (zero to nine).

C2 Get the data

In [188…
```
from sklearn.datasets import load_digits
digits = load_digits()
digits.keys() # the data is in the form of a dictionary, this will let you see the keys
```

Out[188…
```
dict_keys(['data', 'target', 'frame', 'feature_names', 'target_names', 'images', 'DESCR'])
```

In [189…
```
digits['DESCR']
```

Out[189…
```
".. _digits_dataset:\n\nOptical recognition of handwritten digits dataset\n--------------------------------------------------\n\n**Data Set Characteristics:**\n\n:Number of Instances: 1797\n:Number of Attributes: 64\n:Attribute Information: 8x8 image of integer pixels in the range 0..16.\n:Missing Attribute Values: None\n:Creator: E. Alpaydin (alpaydin '@' boun.edu.tr)\n:Date: July; 1998\n\nThis is a copy of the test set of the UCI ML hand-written digits datasets\nhttps://archive.ics.uci.edu/ml/datasets/Optical+Recognition+of+Handwritten+Digits\n\nThe data set contains images of hand-written digits: 10 classes where\neach class refers to a digit.\n\nPreprocessing programs made available by NIST were used to extract\nnormalized bitmaps of handwritten digits from a preprinted form. From a\ntotal of 43 people, 30 contributed to the training set and different 13\nto the test set. 32x32 bitmaps are divided into nonoverlapping blocks of\n4x4 and the number of on pixels are counted in each block. This generates\nan input matrix of 8x8 where each element is an integer in the range\n0..16. This reduces dimensionality and gives invariance to small\ndistortions.\n\nFor info on NIST preprocessing routines, see M. D. Garris, J. L. Blue, G.\nT. Candela, D. L. Dimmick, J. Geist, P. J. Grother, S. A. Janet, and C.\nL. Wilson, NIST Form-Based Handprint Recognition System, NISTIR 5469,\n1994.\n\n.. dropdown:: References\n\n  - C. Kaynak (1995) Methods of Combining Multiple Classifiers and Their\n    Applications to Handwritten Digit Recognition, MSc Thesis, Institute of\n    Graduate Studies in Science and Engineering, Bogazici University.\n  - E. Alpaydin, C. Kaynak (1998) Cascading Classifiers, Kybernetika.\n  - Ken Tang and Ponnuthurai N. Suganthan and Xi Yao and A. Kai Qin.\n    Linear dimensionalityreduction using relevance weighted LDA. School of\n    Electrical and Electronic Engineering Nanyang Technological University.\n    2005.\n  - Claudio Gentile. A New Approximate Maximal Margin Classification\n  Algorithm. NIPS. 2000.\n"
```

In [190…
```
digits.data[1]
```

Out[190…
```
array([ 0.,  0.,  0., 12., 13.,  5.,  0.,  0.,  0.,  0.,  0., 11., 16.,
        9.,  0.,  0.,  0.,  0.,  3., 15., 16.,  6.,  0.,  0.,  0.,  7.,
       15., 16., 16.,  2.,  0.,  0.,  0.,  0.,  1., 16., 16.,  3.,  0.,
        0.,  0.,  0.,  1., 16., 16.,  6.,  0.,  0.,  0.,  0.,  1., 16.,
       16.,  6.,  0.,  0.,  0.,  0.,  0., 11., 16., 10.,  0.,  0.])
```

In [191…
```
digits.images[1]
```

Out[191…
```
array([[ 0.,  0.,  0., 12., 13.,  5.,  0.,  0.],
       [ 0.,  0.,  0., 11., 16.,  9.,  0.,  0.],
       [ 0.,  0.,  3., 15., 16.,  6.,  0.,  0.],
       [ 0.,  7., 15., 16., 16.,  2.,  0.,  0.],
       [ 0.,  0.,  1., 16., 16.,  3.,  0.,  0.],
       [ 0.,  0.,  1., 16., 16.,  6.,  0.,  0.],
       [ 0.,  0.,  1., 16., 16.,  6.,  0.,  0.],
       [ 0.,  0.,  0., 11., 16., 10.,  0.,  0.]])
```

In [196…
```
print(f"Number of samples: {digits.data.shape[0]}")
```

```
Number of samples: 1797
```

The dictionary has:

1. **data**: 1797 samples x 64 features matrix. Each sample is a flattened 8x8 image.
2. **target**: Labels for each sample (digits 0-9), shape (1797,)
3. **target_names**: Array of digit labels [0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
4. **images**: The same data organized as 1797 8x8 matrices for visualization
5. **DESCR**: Text description of the dataset

C3 Explore the data

Question 12

(2 pts)

Check the shape, and head, plot, visualize, and have a look at the data with whatever ways you think will help your task. Comment throughout on observations. There is some code below to look at some sample of digits

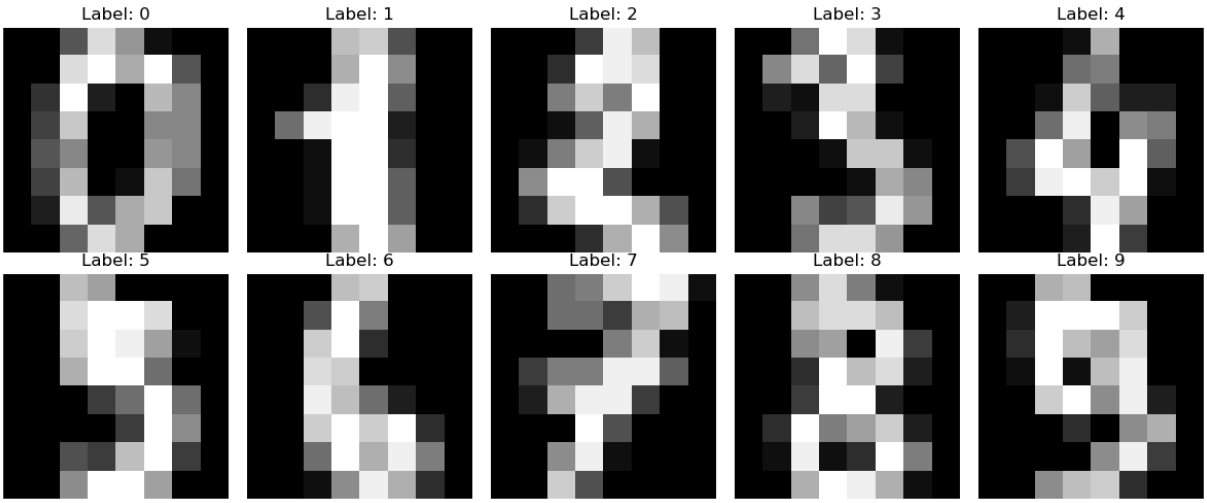
In [197…
```
#Check the shape
print(f"Data shape: {digits.data.shape}")
print(f"Images shape: {digits.images.shape}")
print(f"Target shape: {digits.target.shape}")
```

```
Data shape: (1797, 64)
Images shape: (1797, 8, 8)
Target shape: (1797,)
```

In [198…
```
# Look at the first few samples
print(f"\nFirst sample data:\n{digits.data[0]}")
print(f"First sample target: {digits.target[0]}")
```

```
First sample data:
[ 0.  0.  5. 13.  9.  1.  0.  0.  0.  0. 13. 15. 10. 15.  5.  0.  0.  3.
 15.  2.  0. 11.  8.  0.  0.  4. 12.  0.  0.  8.  8.  0.  0.  5.  8.  0.
  0.  9.  8.  0.  0.  4. 11.  0.  1. 12.  7.  0.  0.  2. 14.  5. 10. 12.
  0.  0.  0.  0.  6. 13. 10.  0.  0.  0.]
First sample target: 0
```

In [199…
```
# 3. Visualize some digits
fig, axes = plt.subplots(2, 5, figsize=(12, 5))
for i, ax in enumerate(axes.flat):
    ax.imshow(digits.images[i], cmap='gray')
    ax.set_title(f'Label: {digits.target[i]}')
    ax.axis('off')
plt.tight_layout()
plt.show()
```



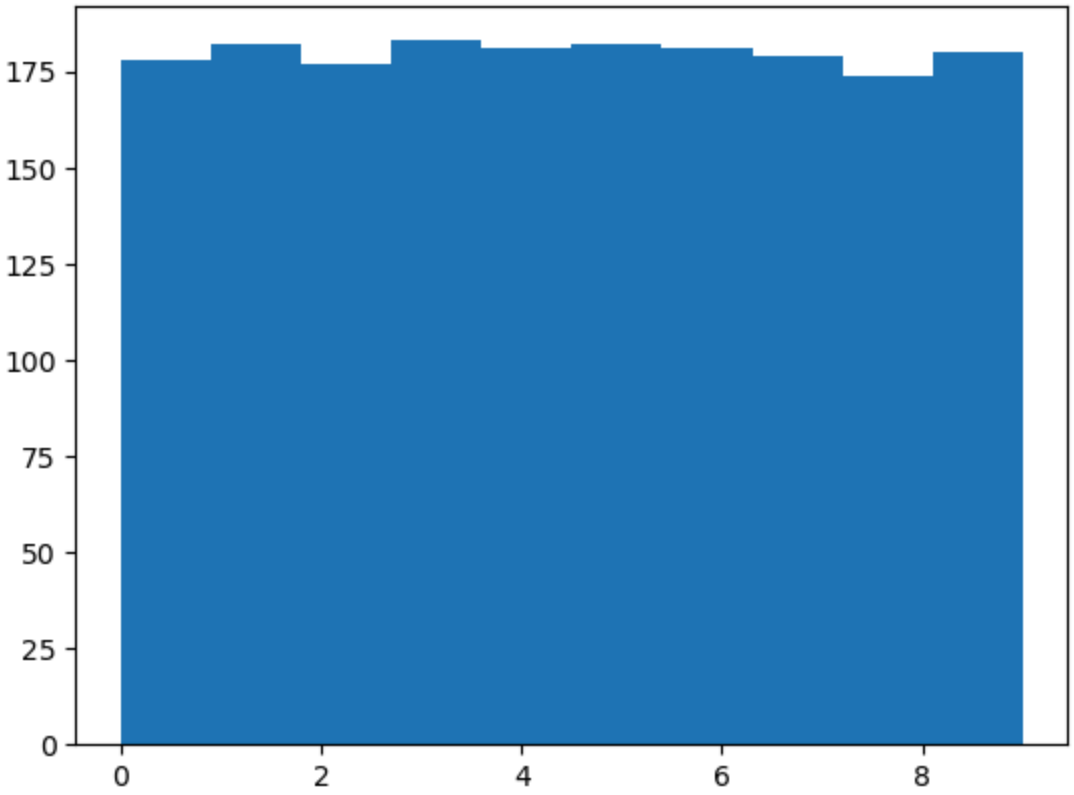
In [200…
```
# 4. Check class distribution
import numpy as np
unique, counts = np.unique(digits.target, return_counts=True)
print(f"\nClass distribution:")
for digit, count in zip(unique, counts):
    print(f"Digit {digit}: {count} samples")
```

```
Class distribution:
Digit 0: 178 samples
Digit 1: 182 samples
Digit 2: 177 samples
Digit 3: 183 samples
Digit 4: 181 samples
Digit 5: 182 samples
Digit 6: 181 samples
Digit 7: 179 samples
Digit 8: 174 samples
Digit 9: 180 samples
```

Classes are well balanced with ~ 180 samples each. The low rsolution (8×8) makes the digits harder to distinguish !

In [201…
```
# Very nice histogram -- the data is balanced amongst all the target
features !
```

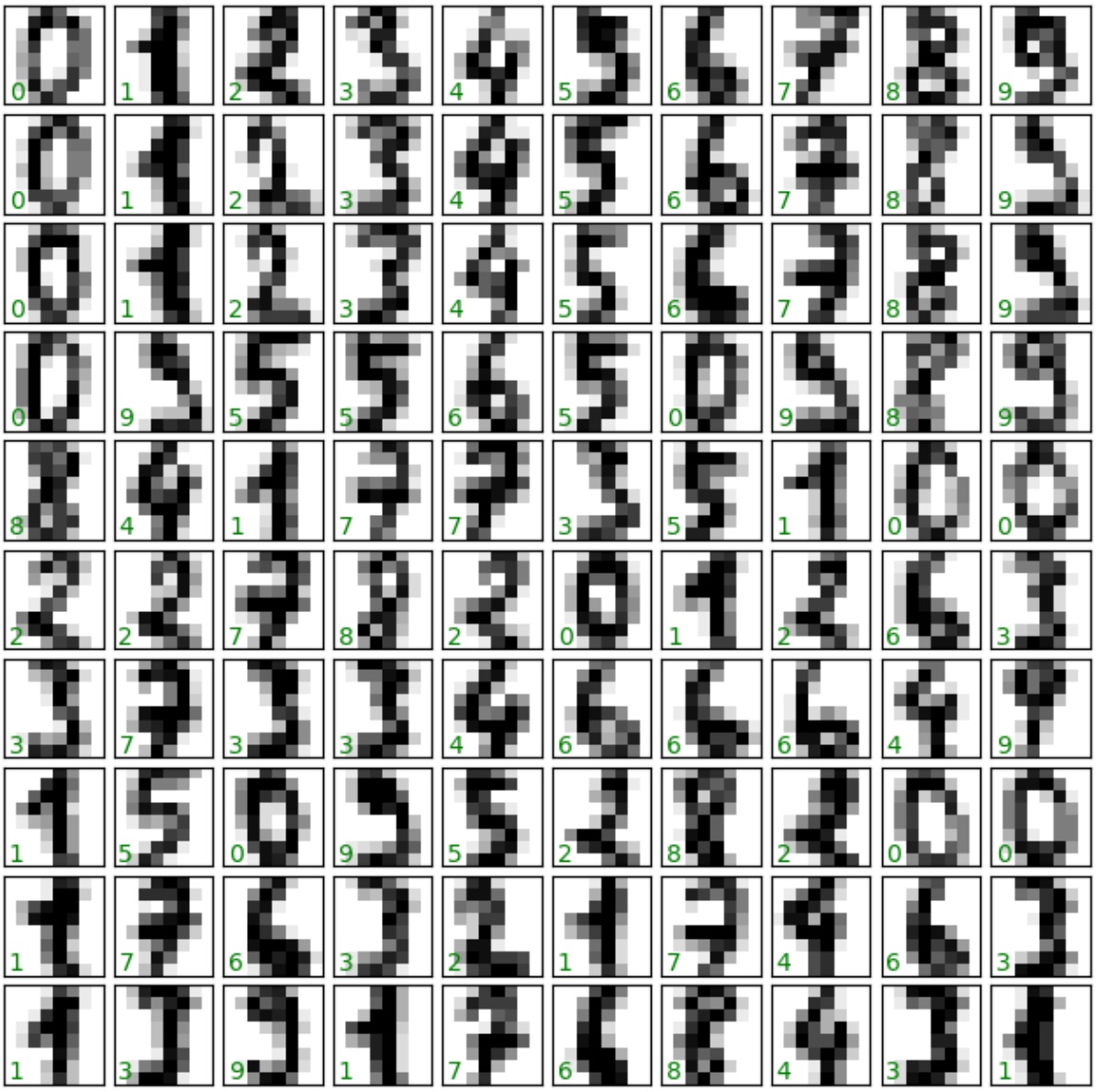
```
plt.hist(digits['target'])
plt.show()
```



In [202…

```
# We can display some of the digits to see what they look like
fig, axes = plt.subplots(10, 10, figsize=(8, 8),
                         subplot_kw={'xticks':[], 'yticks':[]},
                         gridspec_kw=dict(hspace=0.1, wspace=0.1))

for i, ax in enumerate(axes.flat):
    ax.imshow(digits.images[i], cmap='binary', interpolation='nearest')
    ax.text(0.05, 0.05, str(digits.target[i]),
            transform=ax.transAxes, color='green')
```



C4 Prepare Data

For this assignment, the data is already prepared and is part of the digits dictionary. No need to do anything else for this part.

C5 Select model and train

Question 12

(6 pts)

Here, we'll be using the pixels as features and the digit as the target. We will use the random forest classifier from sklearn to model the data.

Remember, in this part, you'll need to :

- Split the data into training, validation, and testing
- Build a random forest classifier
- Train the classifier

In [203…
```
# your code here
# split data
x_train, x_validate, y_train, y_validate =
model_selection.train_test_split(digits['data'], digits['target'],
test_size=0.1)
x_train, x_test, y_train, y_test =
model_selection.train_test_split(x_train, y_train, test_size=0.2)
```

In [206…
```
# Load classifier

from sklearn.ensemble import RandomForestClassifier
```

In [207…
```
clf=RandomForestClassifier(max_depth=7)
clf.fit(x_train,y_train)
```

Out[207…

▾ RandomForestClassifier

```
RandomForestClassifier(max_depth=7)
```

Performance / evaluation metrics

Question 13

(6 pts)

We have mainly been using **accuracy score** as a metric for classification, but that is just one measure of how well our predictions are. For this part, let's compute some additional metrics.

- First, compute the accuracy score of your random forest classifier.
- Compute the confusion matrix and plot it (see lab 1)
- Learn about and compute another classification metric in sklearn: https://scikit-learn.org/stable/modules/model_evaluation.html#classification-metrics

In [209…
```
# enter your code here
y_pred = clf.predict(x_test)
print("accuracy:", accuracy_score(y_test, y_pred))
```

```
accuracy: 0.9537037037037037
```

We get an accuracy score of 95.4%.

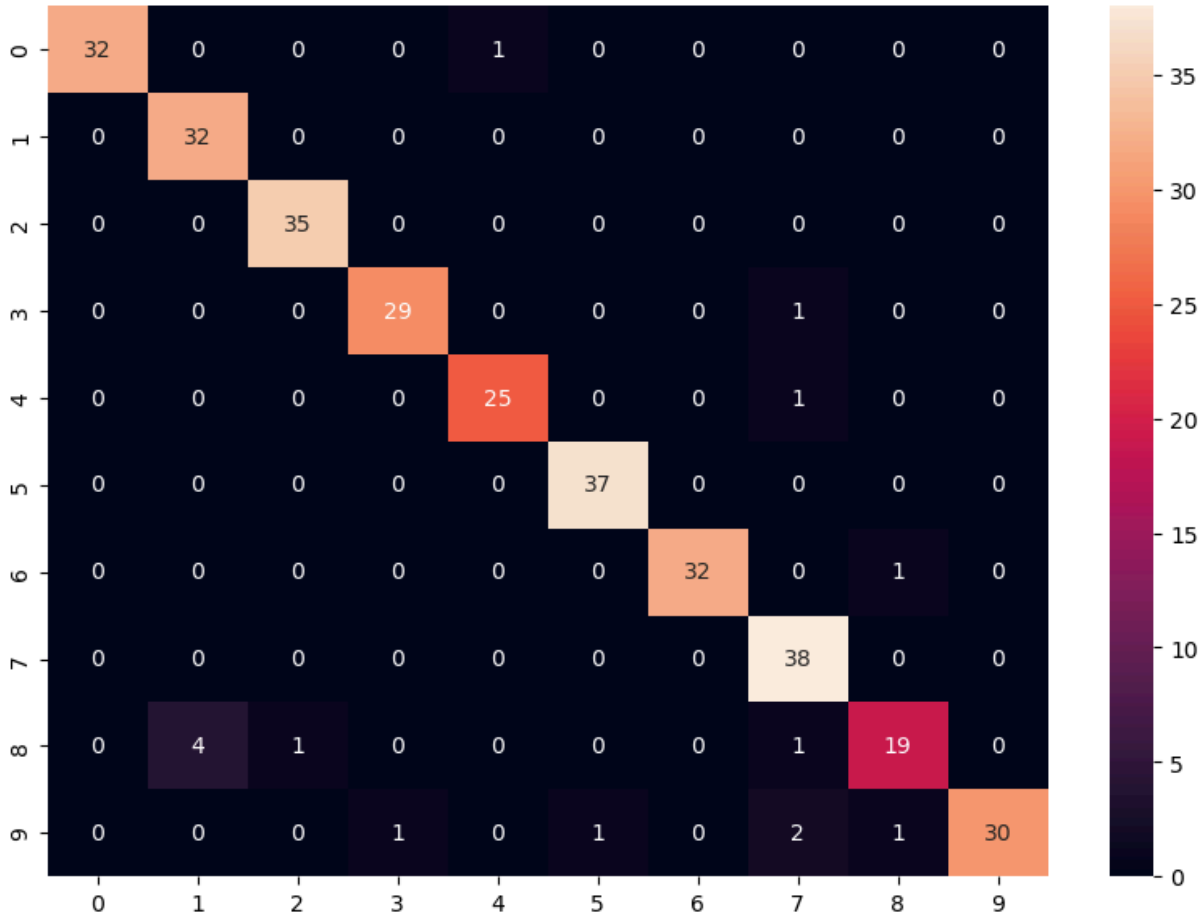
In [210…
```
from sklearn.metrics import confusion_matrix
```

In [211…
```
confusion_matrix(y_test,y_pred)
```

Out[211…
```
array([[32,  0,  0,  0,  1,  0,  0,  0,  0,  0],
       [ 0, 32,  0,  0,  0,  0,  0,  0,  0,  0],
       [ 0,  0, 35,  0,  0,  0,  0,  0,  0,  0],
       [ 0,  0,  0, 29,  0,  0,  0,  1,  0,  0],
       [ 0,  0,  0,  0, 25,  0,  0,  1,  0,  0],
       [ 0,  0,  0,  0,  0, 37,  0,  0,  0,  0],
       [ 0,  0,  0,  0,  0,  0, 32,  0,  1,  0],
       [ 0,  0,  0,  0,  0,  0,  0, 38,  0,  0],
       [ 0,  4,  1,  0,  0,  0,  0,  1, 19,  0],
       [ 0,  0,  0,  1,  0,  1,  0,  2,  1, 30]])
```

In [212…
```
array = confusion_matrix(y_test,y_pred)
plt.figure(figsize = (10,7))
sns.heatmap(array, annot=True)
```

Out[212…
```
<Axes: >
```



In [215…
```
#Let's do classification report
from sklearn.metrics import classification_report

# Get predictions
y_pred = clf.predict(x_test)
```

```
# Print classification report
print(classification_report(y_test, y_pred))

# Or with digit names as labels
print(classification_report(y_test, y_pred, target_names=[str(i) for i in
range(10)]))
```

```
              precision    recall  f1-score   support

           0       1.00      0.97      0.98        33
           1       0.89      1.00      0.94        32
           2       0.97      1.00      0.99        35
           3       0.97      0.97      0.97        30
           4       0.96      0.96      0.96        26
           5       0.97      1.00      0.99        37
           6       1.00      0.97      0.98        33
           7       0.88      1.00      0.94        38
           8       0.90      0.76      0.83        25
           9       1.00      0.86      0.92        35

    accuracy                           0.95       324
   macro avg       0.96      0.95      0.95       324
weighted avg       0.96      0.95      0.95       324

              precision    recall  f1-score   support

           0       1.00      0.97      0.98        33
           1       0.89      1.00      0.94        32
           2       0.97      1.00      0.99        35
           3       0.97      0.97      0.97        30
           4       0.96      0.96      0.96        26
           5       0.97      1.00      0.99        37
           6       1.00      0.97      0.98        33
           7       0.88      1.00      0.94        38
           8       0.90      0.76      0.83        25
           9       1.00      0.86      0.92        35

    accuracy                           0.95       324
   macro avg       0.96      0.95      0.95       324
weighted avg       0.96      0.95      0.95       324
```

INTERPRETATION OF THE RESULTS FROM THE CLASSIFICATION REPORT

1. OVERALL PERFORMANCE : The model correctly classifies 95% of all digits !
2. BEST PERFORMING DIGITS : Digits $0,\ 6,\ 9$ and 1.
3. CHALLENGING DIGITS : 8 has a lower recall of 0.76 - misses 24% of all actual 8's and confuses them with other digits !
4. A **Precision** of 0.96 implies that when the model predicts a digit, it's correct 96% of the time. A **Recall** of 0.95 implies that the model successfully identifies 95% of all actual digits !. An **F1 score** of 0.95 implies a balanced measure showing strong overall performance !

3.6 Fine tune model

Question 14

(6 pts)

The random forest classifier has many hyper-parameters to tune that can make your classification better. Here, you'll conduct some systematic experiments in tuning hyper-parameters to see how they affect the model performance.

- First, read the documentation for the random forest classifier to see the different hyper-parameters https://scikit-learn.org/stable/modules/generated/sklearn.ensemble.RandomForestClassifier.html
- Pick one of the hyper-parameters to tune (some examples are: `n_estimators`, `max_depth`, `min_samples_leaf`, `max_leaf_nodes`)
- Systematically vary one of these hyper-parameters and record your performance metric (for example accuracy score). Plot this relationship.
- If you want to search multiple parameters in a grid, sklearn also has a wrapper function that is very useful called `GridSearchCV` https://scikit-learn.org/stable/modules/generated/sklearn.model_selection.GridSearchCV.html
- Discuss the results of your experiments

Below I choose the hyperparamter n_estimators to vary -- I calculate the accuracy as I vary this hyperparameter. I plot this dependence and glean what the most optimal choice if !

In [226…
```
# your code here
estimators=np.linspace(20,300,11)
accuracies=[]
for i in estimators:
    clf = RandomForestClassifier(int(i),max_depth=6)
    clf.fit(x_train, y_train)
    y_pred=clf.predict(x_test)
    accuracies.append(accuracy_score(y_test,y_pred))
```

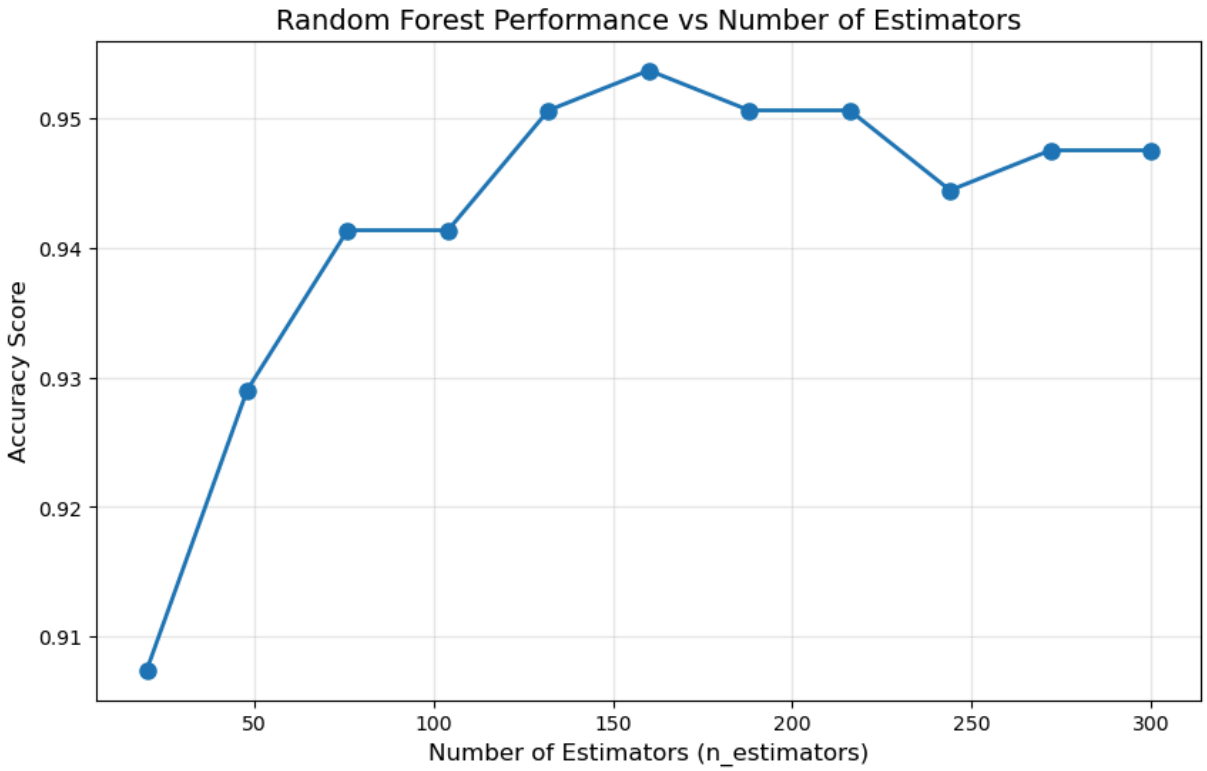
In [227…
```
accuracies
```

Out[227…
```
[0.9074074074074074,
 0.9290123456790124,
 0.941358024691358,
 0.941358024691358,
 0.9506172839506173,
 0.9537037037037037,
 0.9506172839506173,
 0.9506172839506173,
 0.9444444444444444,
 0.9475308641975309,
 0.9475308641975309]
```

In [229…
```
# Citation -- Used Claude to help me plot the data !
plt.figure(figsize=(10, 6))
plt.plot(estimators, accuracies, marker='o', linewidth=2, markersize=8)
plt.xlabel('Number of Estimators (n_estimators)', fontsize=12)
plt.ylabel('Accuracy Score', fontsize=12)
plt.title('Random Forest Performance vs Number of Estimators', fontsize=14)
plt.grid(True, alpha=0.3)
plt.show()

# Print the results
for n_est, acc in zip(estimators, accuracies):
    print(f"n_estimators={int(n_est):3d} → Accuracy: {acc:.4f}")

# Find best
best_idx = np.argmax(accuracies)
print(f"\nBest performance: n_estimators={int(estimators[best_idx])} with
accuracy={accuracies[best_idx]:.4f}")
```



```
n_estimators= 20 → Accuracy: 0.9074
n_estimators= 48 → Accuracy: 0.9290
n_estimators= 76 → Accuracy: 0.9414
n_estimators=104 → Accuracy: 0.9414
n_estimators=132 → Accuracy: 0.9506
n_estimators=160 → Accuracy: 0.9537
n_estimators=188 → Accuracy: 0.9506
n_estimators=216 → Accuracy: 0.9506
n_estimators=244 → Accuracy: 0.9444
n_estimators=272 → Accuracy: 0.9475
n_estimators=300 → Accuracy: 0.9475

Best performance: n_estimators=160 with accuracy=0.9537
```

Hyperparameter Tuning Results

I varied the number of trees (n_estimators) from 20 to 300 while keeping max_depth=10 constant.

Conclusion: The optimal number of estimators appears to be around 160, which balances performance and computational cost. More trees improve accuracy initially through ensemble averaging, but eventually reach a point of diminishing returns where additional trees don't significantly improve predictions.

GRID SEARCH CV RESULTS !

Vary over the max_depth, n_estimators and min_samples_leaf

In [234…
```
rf_grid=RandomForestClassifier()
gr_space = {
    'max_depth' : [3,5,7,10],
    'n_estimators' : [50,100,150,200,250],
    'min_samples_leaf' : [2,3,5,6,7]
}
```

In [231…
```
from sklearn.model_selection import GridSearchCV
grid=GridSearchCV(rf_grid,gr_space,cv=3,scoring='accuracy')
model_grid=grid.fit(x_train,y_train)
```

In [236…
```
# The most optimal model
print(model_grid.best_params_)
```

```
{'max_depth': 10, 'min_samples_leaf': 2, 'n_estimators': 250}
```

In [237…
```
print(model_grid.best_score_)
```

```
0.9636504253673627
```

A Grid Search CV increases our accuracy from 95.4% to 96.4% !!

3.7 Present solution

Question 15

(4 pts)

Using the testing data that you left out in the beginning, compute your final performance metrics. Also plot the final confusion matrix. Discuss where your model is good and where the model can still be improved.

The optimal model we use is the one given by **GridSearchCV**

On the testing data !

In [ ]:
```
# your code here
clf_optimal =
RandomForestClassifier(n_estimators=250,max_depth=10,min_samples_leaf=2)
clf_optimal.fit(x_train, y_train)
y_pred=clf_optimal.predict(x_test)
print(accuracy_score(y_test,y_pred))
```

```
0.9691358024691358
```

On the validation data set !

In [ ]:
```
# your code here

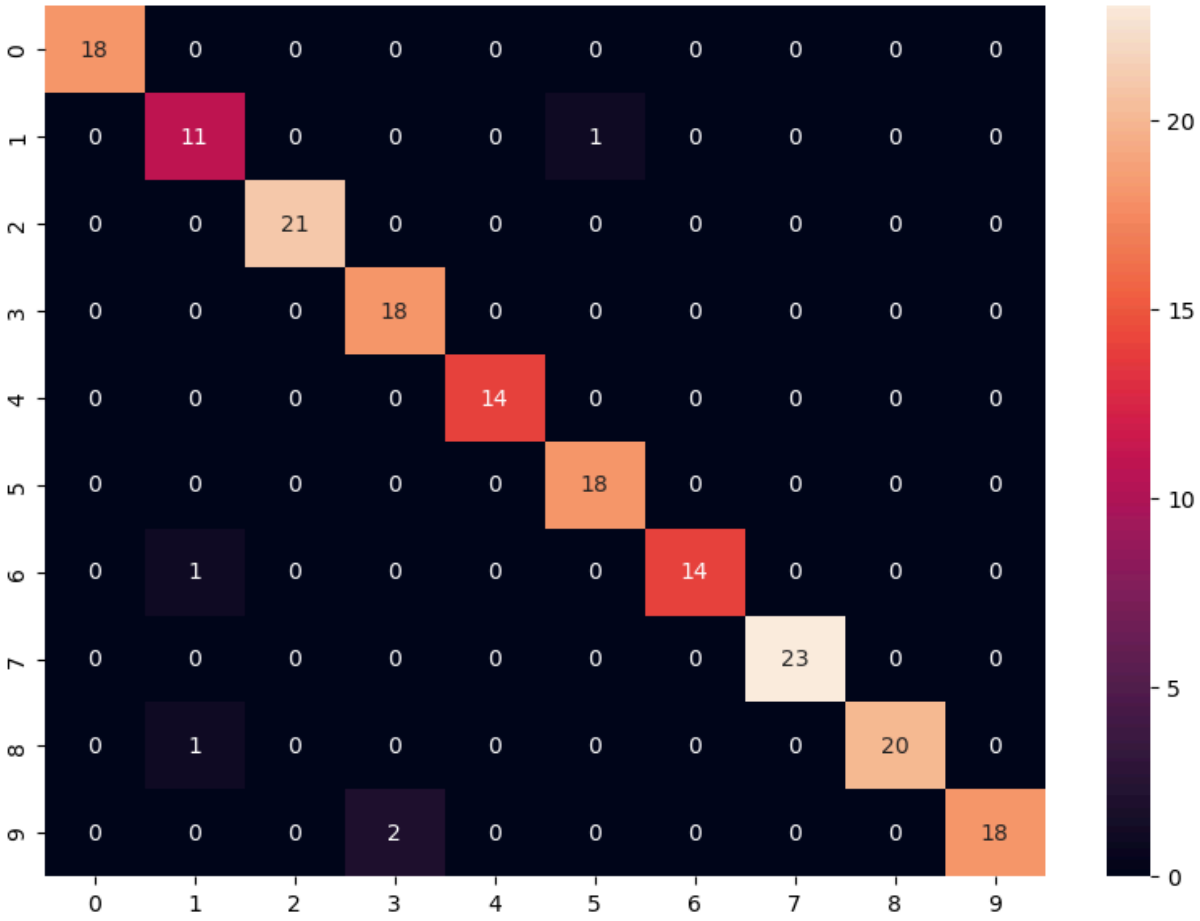
clf_optimal =
RandomForestClassifier(n_estimators=250,max_depth=10,min_samples_leaf=2)
clf_optimal.fit(x_train, y_train)
y_pred=clf_optimal.predict(x_validate)
print(accuracy_score(y_validate,y_pred))
```

```
0.9722222222222222
```

Accuracy score of 97.2% on the validation data set !!

In [243…
```
array = confusion_matrix(y_validate,y_pred)
plt.figure(figsize = (10,7))
sns.heatmap(array, annot=True)
```

Out[243…
```
<Axes: >
```



Final Model Performance

Optimal Random Forest Model Configuration obtained via GridSearchCV:

- n_estimators: 250
- max_depth: 10
- min_samples_leaf: 2

Performance on Validation Set:

- **Accuracy: 97.22% -** The model correctly classifies 97.22% of digits

Strengths (where model is good):

- High accuracy on digits with distinct shapes -- this is clear from the confusion matrix !

Areas for Improvement:

- From the off-diagonal confusion matrix values Digit 3 is sometimes confused with digit 9 (2 instances), for example.
- Could potentially improve by:
  - Adding more training data for problematic digits

- Further hyperparameter tuning (trying more depth values)

Overall: The model achieves strong performance (97.22%) (though some challenging digit pairs could benefit from additional refinement) !!

3.8. Launch, Monitor and Maintain

Nothing to here for this lab.

Part D - Machine Learning in Particle Physics

D1 - The Big Picture

In this part, you will build a random forest machine learning model to recongnize types creating of top-quarks in a particle accelarator based on the observed products. We can use the decay products, energy, and momentum vectors to infer the presense of top-quarks. Beyond Standard Model Particle physics models often predict new types of particles so knowing whether the number of top-quarks (which have very high energy compared to other quarks) are as expected can be important for identifying when the Standard Model breaks down.

Before you begin this experiment, read more about the dataset in Chapter 4, Section 4.1 of the Machine Learning for Physics and Astronomy in the PDF in the lab directory (also on Bruinlearn).

D2 - Get the Data

The dataset is included here in `ParticleID_features.csv`, the target is in `'ParticleID_labels.txt`. Our goal is to differentiate between the `4-top` events (Interactions of 4 top quark) from the `ttbar` events (interaction of top and anti-top quarks background events).

In [245…
```
## get the data

import pandas as pd
features = pd.read_csv('ParticleID_features.csv')
targets = pd.read_csv('ParticleID_labels.txt',header=None)
```

In [246…
```
features
```

Out [246...

	ID	MET	METphi	Type_1	P1	P2	P3	P4
0	0	62803.5	-1.810010	j	137571.0	128444.0	-0.345744	-0.307112
1	1	57594.2	-0.509253	j	161529.0	80458.3	-1.318010	1.402050
2	2	82313.3	1.686840	b	167130.0	113078.0	0.937258	-2.068680
3	3	30610.8	2.617120	j	112267.0	61383.9	-1.211050	-1.457800
4	4	45153.1	-2.241350	j	178174.0	100164.0	1.166880	-0.018721
...	...	...	...	...	...	...	...	...
4995	4995	269074.0	-1.274730	j	495577.0	362590.0	-0.791914	1.671250
4996	4996	12385.8	0.986871	j	258932.0	133559.0	-1.276540	2.970100
4997	4997	32762.8	3.057630	b	122222.0	79947.8	0.983920	-0.399231
4998	4998	104474.0	-1.875250	b	791028.0	457589.0	1.141530	2.934810
4999	4999	65623.8	-0.817535	b	145360.0	138746.0	0.236765	2.277120

5000 rows × 68 columns

D3 - Explore the data

In this lab, you have the freedom to decide what features you would like to use to train your model and how to clean and inpute your data.

To help guide you, be sure to answer the following questions about the data. Read Chapter 4, Section 4.1 and 4.3 of the included PDF to learn more and get ideas of how to deal with cleaning the data and selecting good features to use. This chapter also has helpful pandas and sklearn code that will help make some of these tasks easier. Use the `features.describe()` to see the statistics of the columns of the `features` table.

Question 16

(4 pts)

1. What are the columns and what do they mean?
2. Why do some columns have more NaN's than others?
3. Examine your target classes. What is the distribution of the number of training samples for the two classes?
4. Make plots to explore some of the features that you are interested in using.

1. **COLUMNS AND MEANINGS**

- 68 total columns representing collision event data

- MET, METphi : Missing energy (amplitude and angle, respectively) from the neutrinos
- Type_N : Particle types (j=jet, b=b-jet, e$\pm$ = electron/positron, m=muon, g= photon)
- P($4N-3$ to $4N$) : 4-vector for each particle (energy, transverse momentum, pseudorapidity, azimuthal angle)

2. **MISSING DATA**

- Different collisions produce different numberso of particles
- 100% of events have particles $1-3$.
- Only $1-2\%$ of events produce 13 particles

In [86]:
```
# the shape of the data frame - 5000 data points and 68 columns
print(features.shape)
print(targets.shape)
```

```
(5000, 68)
(5000, 1)
```

In [85]:
```
# what the two possible outputs can be
targets
```

Out[85]:

	0
0	ttbar
1	ttbar
2	ttbar
3	ttbar
4	ttbar
...	...
4995	4top
4996	ttbar
4997	ttbar
4998	4top
4999	ttbar

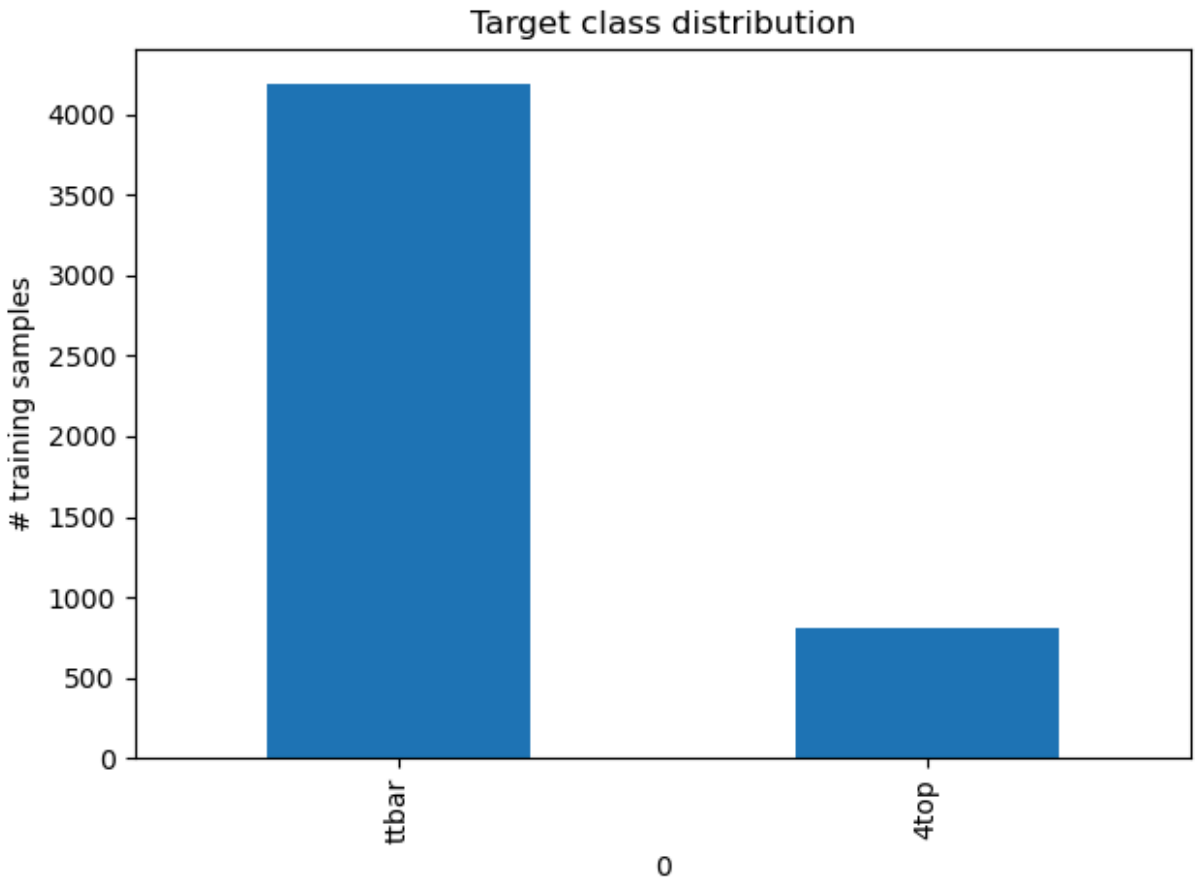
5000 rows × 1 columns

In [ ]:
```
# number of ttbar events and 4-top events
targets.value_counts()
```

Out[ ]:
```
0
ttbar    4189
4top      811
Name: count, dtype: int64
```

The data set is imbalalanced -- the number of 4-top events is a lot less than the number of ttbar events !

In [87]:
```
plt.figure()
class_counts = targets[0].value_counts()
class_counts.plot(kind="bar")
plt.ylabel("# training samples")
plt.title("Target class distribution")
plt.tight_layout()
plt.show()
```



3. **TARGET DISTRIBUTION**

- 16 are 4-top events -- these are the rare, interesting events
- 84% are ttbar events -- this is the common background
- The data is imbalanced !

In [73]:
```
features
```

Out[73]:

	ID	MET	METphi	Type_1	P1	P2	P3	P4
0	0	62803.5	-1.810010	j	137571.0	128444.0	-0.345744	-0.307112
1	1	57594.2	-0.509253	j	161529.0	80458.3	-1.318010	1.402050
2	2	82313.3	1.686840	b	167130.0	113078.0	0.937258	-2.068680
3	3	30610.8	2.617120	j	112267.0	61383.9	-1.211050	-1.457800
4	4	45153.1	-2.241350	j	178174.0	100164.0	1.166880	-0.018721
...	...	...	...	...	...	...	...	...
4995	4995	269074.0	-1.274730	j	495577.0	362590.0	-0.791914	1.671250
4996	4996	12385.8	0.986871	j	258932.0	133559.0	-1.276540	2.970100
4997	4997	32762.8	3.057630	b	122222.0	79947.8	0.983920	-0.399231
4998	4998	104474.0	-1.875250	b	791028.0	457589.0	1.141530	2.934810
4999	4999	65623.8	-0.817535	b	145360.0	138746.0	0.236765	2.277120

5000 rows × 68 columns

In [5]:
```
features.describe()
```

Out[5]:

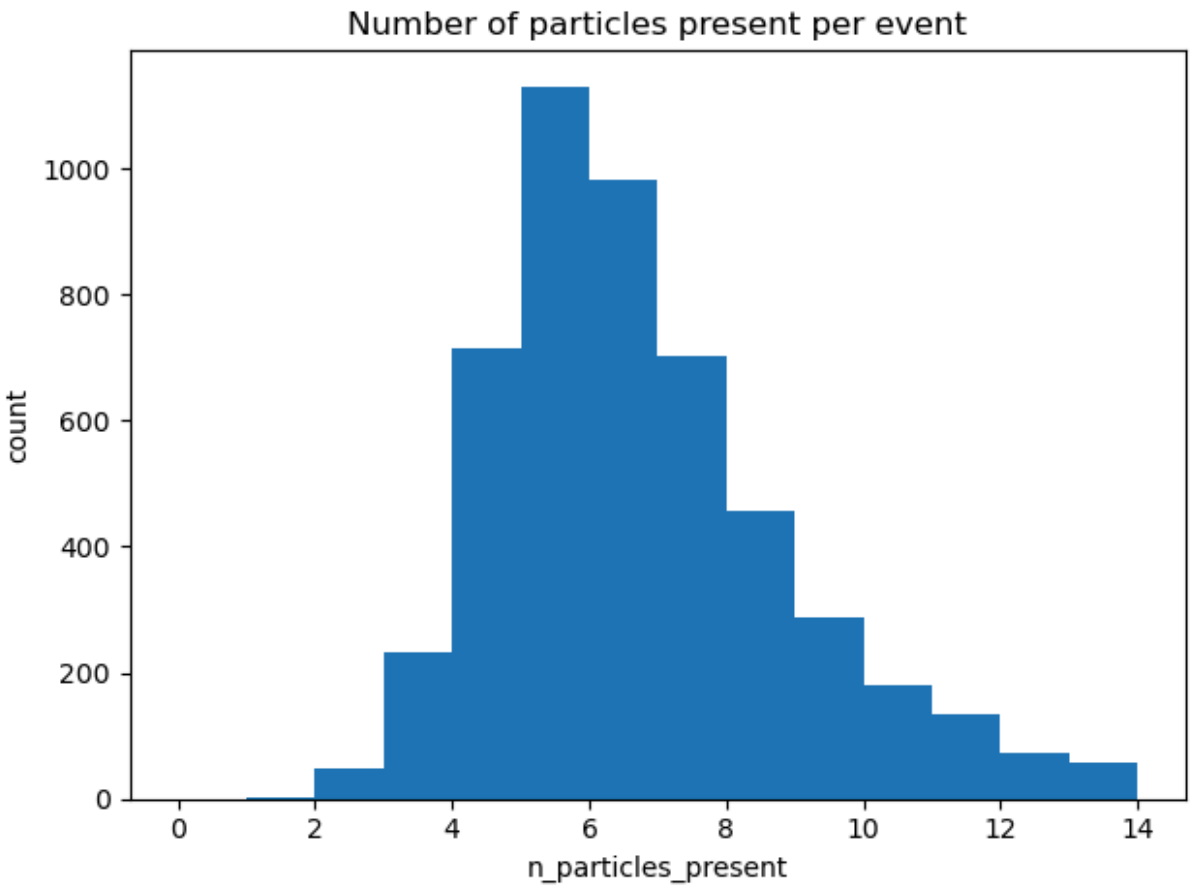
	ID	MET	METphi	P1	P2	
count	5000.000000	5000.000000	5000.000000	5.000000e+03	5.000000e+03	500
mean	2499.500000	64071.074332	-0.028916	3.301357e+05	1.540486e+05	-
std	1443.520003	60525.122480	1.819257	3.068202e+05	1.149469e+05	
min	0.000000	290.756000	-3.141010	3.857940e+04	2.825400e+04	-
25%	1249.750000	24352.375000	-1.619905	1.369522e+05	8.883690e+04	-
50%	2499.500000	46814.400000	-0.055612	2.263525e+05	1.182015e+05	-
75%	3749.250000	83032.350000	1.537323	4.077158e+05	1.771265e+05	
max	4999.000000	692674.000000	3.141130	3.186360e+06	1.276710e+06	

8 rows × 55 columns

In [104…
```
features["n_particles_present"] = features[type_cols].notna().sum(axis=1)
features["n_particles_present"].head()


plt.figure()
plt.hist(features["n_particles_present"], bins=range(0, 15))
plt.xlabel("n_particles_present")
plt.ylabel("count")
plt.title("Number of particles present per event")
```

```
plt.tight_layout()
plt.show()
```



In [105…
```
at_least = features["n_particles_present"].value_counts().sort_index()
[::-1].cumsum()[::-1]
# at_least[k] = #events with n_particles_present >= k
at_least
```

Out[105…
```
n_particles_present
1     5000
2     4997
3     4950
4     4717
5     4002
6     2871
7     1889
8     1186
9      729
10     442
11     261
12     127
13      56
Name: count, dtype: int64
```

4. FEATURE VISUALIZATION

In [249…
```
## Citation – Used Claude to help me with these visualizations !
# Create comprehensive visualizations
```

```
fig, axes = plt.subplots(3, 3, figsize=(18, 15))
fig.suptitle('ParticleID Features Exploration', fontsize=18,
fontweight='bold')

# 1. MET distribution
axes[0, 0].hist(features['MET'], bins=50, edgecolor='black', alpha=0.7,
color='steelblue')
axes[0, 0].set_xlabel('MET (Missing Transverse Energy)', fontsize=11)
axes[0, 0].set_ylabel('Frequency', fontsize=11)
axes[0, 0].set_title('Missing Energy Distribution', fontsize=12,
fontweight='bold')
axes[0, 0].grid(alpha=0.3)

# 2. METphi distribution
axes[0, 1].hist(features['METphi'], bins=50, edgecolor='black', alpha=0.7,
color='orange')
axes[0, 1].set_xlabel('METphi (Azimuthal Angle)', fontsize=11)
axes[0, 1].set_ylabel('Frequency', fontsize=11)
axes[0, 1].set_title('MET Angle Distribution', fontsize=12,
fontweight='bold')
axes[0, 1].grid(alpha=0.3)

# 3. First particle energy distribution
axes[0, 2].hist(features['P1'].dropna(), bins=50, edgecolor='black',
alpha=0.7, color='green')
axes[0, 2].set_xlabel('P1 (Energy of 1st particle)', fontsize=11)
axes[0, 2].set_ylabel('Frequency', fontsize=11)
axes[0, 2].set_title('First Particle Energy', fontsize=12,
fontweight='bold')
axes[0, 2].grid(alpha=0.3)

# 4. Particle type distribution (first particle)
type_counts = features['Type_1'].value_counts()
axes[1, 0].bar(type_counts.index, type_counts.values, edgecolor='black',
color='purple', alpha=0.7)
axes[1, 0].set_xlabel('Particle Type', fontsize=11)
axes[1, 0].set_ylabel('Count', fontsize=11)
axes[1, 0].set_title('First Particle Type Distribution', fontsize=12,
fontweight='bold')
axes[1, 0].grid(axis='y', alpha=0.3)

# 5. Number of particles per event
particle_counts = []
for idx in range(len(features)):
    count = 0
    for i in range(1, 14):
        if pd.notna(features.loc[idx, f'Type_{i}']):
            count += 1
    particle_counts.append(count)

axes[1, 1].hist(particle_counts, bins=range(1, 15), edgecolor='black',
alpha=0.7, color='red')
axes[1, 1].set_xlabel('Number of Particles Produced', fontsize=11)
axes[1, 1].set_ylabel('Number of Events', fontsize=11)
axes[1, 1].set_title('Particles Per Event', fontsize=12, fontweight='bold')
axes[1, 1].set_xticks(range(1, 14))
```

```
axes[1, 1].grid(alpha=0.3)

# 6. Energy vs Transverse Momentum (first particle)
axes[1, 2].scatter(features['P1'], features['P2'], alpha=0.3, s=10,
color='teal')
axes[1, 2].set_xlabel('Energy (P1)', fontsize=11)
axes[1, 2].set_ylabel('Transverse Momentum (P2)', fontsize=11)
axes[1, 2].set_title('Energy vs Momentum (1st Particle)', fontsize=12,
fontweight='bold')
axes[1, 2].grid(alpha=0.3)

# 7. Transverse momentum distribution
axes[2, 0].hist(features['P2'].dropna(), bins=50, edgecolor='black',
alpha=0.7, color='coral')
axes[2, 0].set_xlabel('P2 (Transverse Momentum)', fontsize=11)
axes[2, 0].set_ylabel('Frequency', fontsize=11)
axes[2, 0].set_title('Transverse Momentum Distribution', fontsize=12,
fontweight='bold')
axes[2, 0].grid(alpha=0.3)

# 8. Pseudorapidity distribution
axes[2, 1].hist(features['P3'].dropna(), bins=50, edgecolor='black',
alpha=0.7, color='gold')
axes[2, 1].set_xlabel('P3 (Pseudorapidity η)', fontsize=11)
axes[2, 1].set_ylabel('Frequency', fontsize=11)
axes[2, 1].set_title('Pseudorapidity Distribution', fontsize=12,
fontweight='bold')
axes[2, 1].grid(alpha=0.3)

# 9. Comparison of particle types across first 4 particles
type_data = []
for i in range(1, 5):
    counts = features[f'Type_{i}'].value_counts()
    for particle_type, count in counts.items():
        type_data.append({'Particle_Number': f'Particle {i}', 'Type':
particle_type, 'Count': count})

type_df = pd.DataFrame(type_data)
particle_types = ['j', 'b', 'e+', 'e-', 'm+', 'm-', 'g']
colors_map = {'j': 'steelblue', 'b': 'orange', 'e+': 'green', 'e-': 'red',
              'm+': 'purple', 'm-': 'brown', 'g': 'pink'}

x = np.arange(4)
width = 0.12
for idx, ptype in enumerate(particle_types):
    subset = type_df[type_df['Type'] == ptype]
    if len(subset) > 0:
        counts = [subset[subset['Particle_Number'] == f'Particle {i}']
['Count'].values[0]
                  if len(subset[subset['Particle_Number'] == f'Particle
{i}']) > 0 else 0
                  for i in range(1, 5)]
        axes[2, 2].bar(x + idx*width, counts, width, label=ptype,
                       color=colors_map.get(ptype, 'gray'),
edgecolor='black')
```

```
axes[2, 2].set_xlabel('Particle Position', fontsize=11)
axes[2, 2].set_ylabel('Count', fontsize=11)
axes[2, 2].set_title('Particle Types Across First 4 Positions',
fontsize=12, fontweight='bold')
axes[2, 2].set_xticks(x + width * 3)
axes[2, 2].set_xticklabels(['1st', '2nd', '3rd', '4th'])
axes[2, 2].legend(fontsize=9)
axes[2, 2].grid(axis='y', alpha=0.3)

plt.tight_layout()
plt.show()

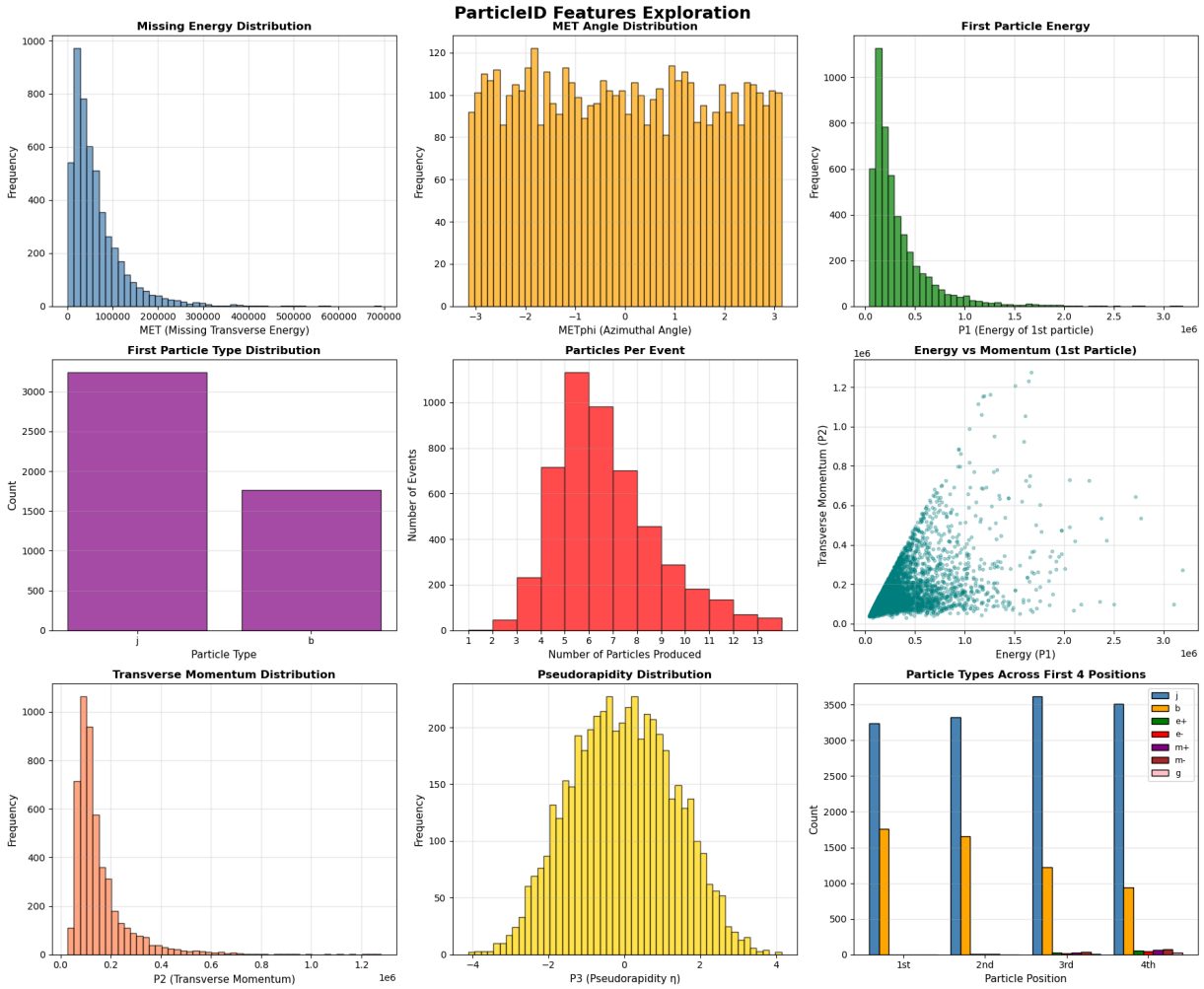
# Additional detailed plots
print("Creating additional plots...")

# Missing data visualization
fig, ax = plt.subplots(figsize=(14, 6))
missing_data = features.isnull().sum()
missing_data = missing_data[missing_data > 0].sort_values(ascending=False)
ax.bar(range(len(missing_data)), missing_data.values, edgecolor='black',
color='darkred', alpha=0.7)
ax.set_xlabel('Feature Index', fontsize=12)
ax.set_ylabel('Number of Missing Values', fontsize=12)
ax.set_title('Missing Values by Feature', fontsize=14, fontweight='bold')
ax.grid(axis='y', alpha=0.3)
plt.tight_layout()
plt.show()

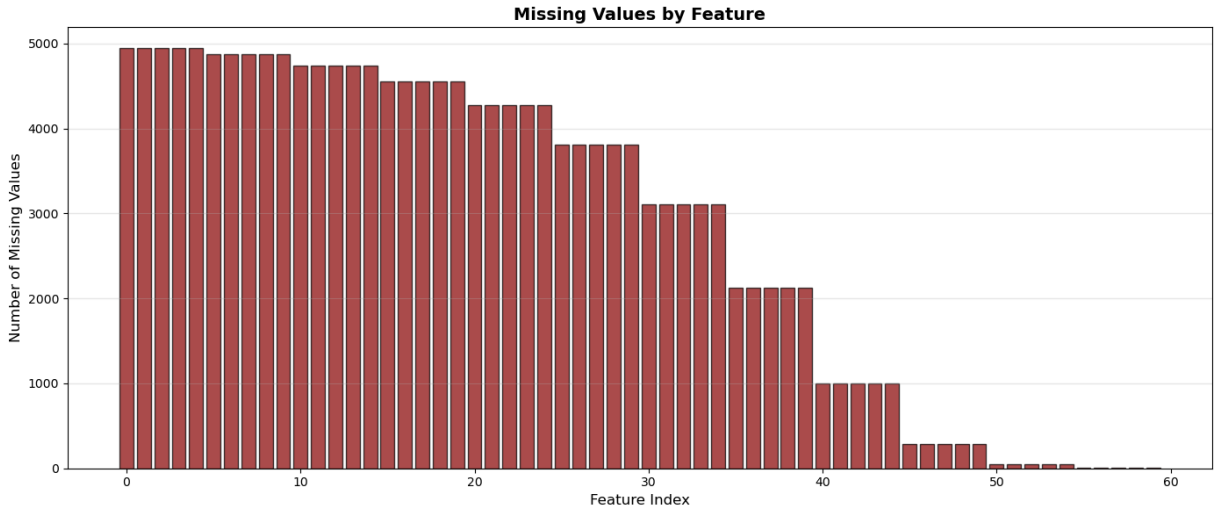
# Boxplots for first 4 particles' energies
fig, axes = plt.subplots(2, 2, figsize=(14, 10))
fig.suptitle('Energy and Momentum Distributions (First 4 Particles)',
fontsize=16, fontweight='bold')

for i in range(1, 5):
    ax = axes[(i-1)//2, (i-1)%2]
    particle_data = features[[f'P{4*i-3}', f'P{4*i-2}']].dropna()
    ax.boxplot([particle_data[f'P{4*i-3}'], particle_data[f'P{4*i-2}']],
               labels=['Energy', 'Momentum'])
    ax.set_ylabel('Value', fontsize=11)
    ax.set_title(f'Particle {i}', fontsize=12, fontweight='bold')
    ax.grid(alpha=0.3)

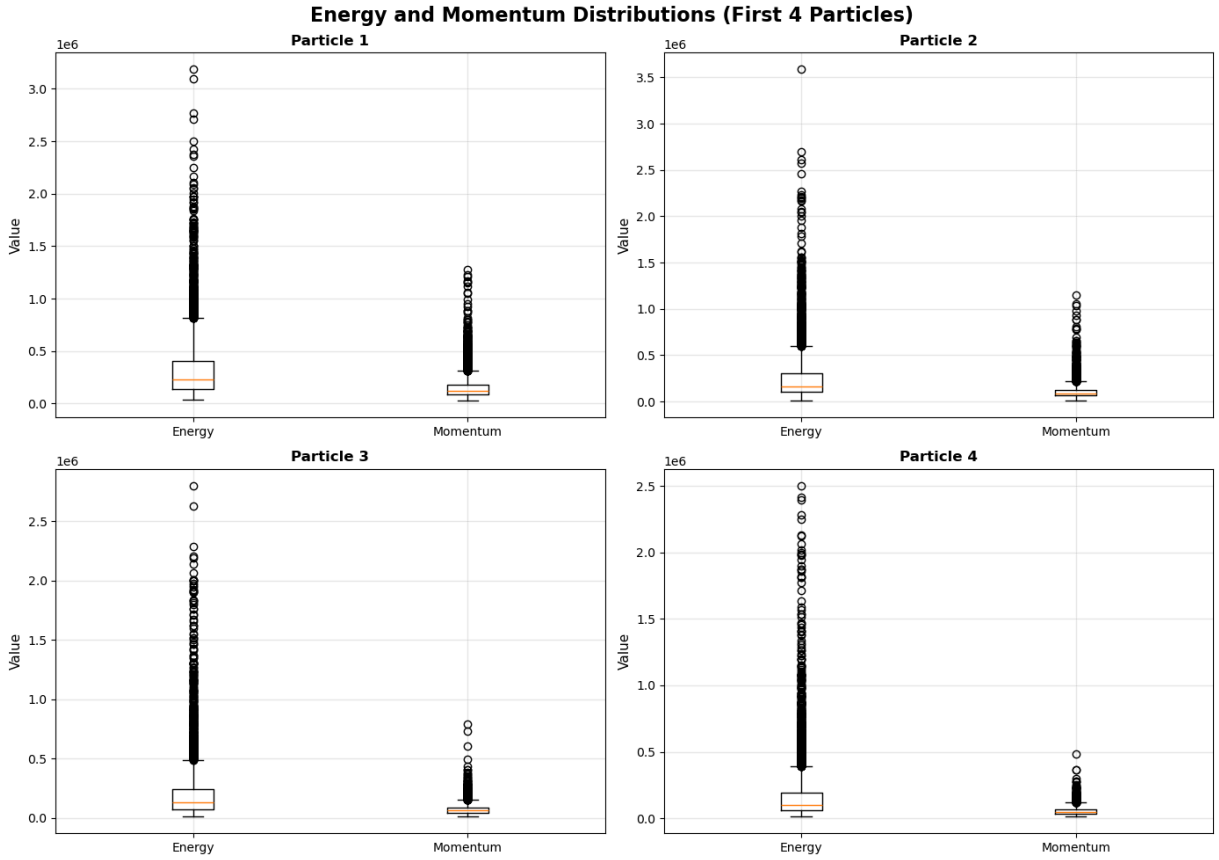
plt.tight_layout()
plt.show()
```




```
Creating additional plots...
```



```
/var/folders/ww/jyhpvg8x3md06rt6f3f93jw00000gn/T/ipykernel_65216/3393820904.
py:128: MatplotlibDeprecationWarning: The 'labels' parameter of boxplot() ha
s been renamed 'tick_labels' since Matplotlib 3.9; support for the old name
will be dropped in 3.11.
  ax.boxplot([particle_data[f'P{4*i-3}'], particle_data[f'P{4*i-2}']],
/var/folders/ww/jyhpvg8x3md06rt6f3f93jw00000gn/T/ipykernel_65216/3393820904.
py:128: MatplotlibDeprecationWarning: The 'labels' parameter of boxplot() ha
s been renamed 'tick_labels' since Matplotlib 3.9; support for the old name
will be dropped in 3.11.
  ax.boxplot([particle_data[f'P{4*i-3}'], particle_data[f'P{4*i-2}']],
/var/folders/ww/jyhpvg8x3md06rt6f3f93jw00000gn/T/ipykernel_65216/3393820904.
py:128: MatplotlibDeprecationWarning: The 'labels' parameter of boxplot() ha
s been renamed 'tick_labels' since Matplotlib 3.9; support for the old name
will be dropped in 3.11.
  ax.boxplot([particle_data[f'P{4*i-3}'], particle_data[f'P{4*i-2}']],
/var/folders/ww/jyhpvg8x3md06rt6f3f93jw00000gn/T/ipykernel_65216/3393820904.
py:128: MatplotlibDeprecationWarning: The 'labels' parameter of boxplot() ha
s been renamed 'tick_labels' since Matplotlib 3.9; support for the old name
will be dropped in 3.11.
  ax.boxplot([particle_data[f'P{4*i-3}'], particle_data[f'P{4*i-2}']],
```



D4 - Prepare the Data

Here, you'll be selecting the features and creating your training, testing, and validating data. Again, Chapter 4, Section 4.1 and 4.3 will be helpful for this.

Question 17

(6 pts)

1. Choose at least 5 features to use in your random forest model and justfy why you think these are helpful features.
2. Do you need to clean these features? Are there invalid or missing values that you need to deal with? If so describe your method for dealing with them.
3. Split your dataset into training, testing, and validation. You are free to choose what fraction you would like to use for each.

1. Feature Selection

I selected 10 features based on the underlying particle physics of top quark decay. These features directly capture the fundamental difference between ttbar (2 tops) and 4-top (4 tops) events.

Background:

Each top quark decays via: **t → b + W**

- Produces 1 **b-quark** (detected as b-jet)
- W boson decays either:
  - **Hadronically:** W → 2 jets (quarks)
  - **Leptonically:** W → lepton + neutrino (MET)

4-top events produce **4 top quarks** vs **2 top quarks** in ttbar, resulting in more of everything.

Selected Features & Justifications:

I mention averages here in this markdown -- if you look at the code cell below, I calculate these averages explicitly

a. n_particles (Total number of detected particles)

- **MOST discriminative feature**
- 4-top: avg 9.4 particles vs ttbar: avg 5.6 particles
- Directly measures decay multiplicity
- Minimal missing data issues (counts what's detected)

Justification: 4 tops → 4 b-quarks + 4 W bosons → more final state particles than 2 tops

b. n_bjets (Number of b-jets)

- **Second most discriminative**
- 4-top: avg 2.4 b-jets vs ttbar: avg 1.2 b-jets

- Clear theoretical prediction: 4 tops produce 4 b-quarks

Justification: Each top directly produces one b-quark, so 4-top events should have ~2x more b-jets than ttbar

c. n_jets (Number of regular jets)

- **Strong discriminator**
- 4-top: avg 6.5 jets vs ttbar: avg 4.1 jets
- Captures hadronic W decay products

Justification: More W bosons → more hadronic decays → more jets

d. n_leptons (Number of leptons: e±, μ±)

- Captures leptonic W decay channel
- 4-top: avg 0.44 leptons vs ttbar: avg 0.28 leptons
- Independent information from jets/b-jets

Justification: More W bosons → more leptonic decays possible. Leptons are cleanly detected and help identify leptonic decay channels.

e. n_electrons (Number of electrons/positrons)

- More specific than n_leptons
- Electrons have different detector signatures than muons
- Provides granular information about lepton types

Justification: W → e + ν_e occurs ~11% of the time. Breaking down lepton types helps model learn specific decay patterns.

f. n_muons (Number of muons)

- Complementary to n_electrons
- Muons penetrate detector differently (longer tracks)
- Independent information about leptonic decays

Justification: W → μ + νμ occurs ~11% of the time. Muons have distinctive signatures that the model can learn.

g. n_photons (Number of photons)

- Captures radiative effects and rare decay modes
- Though rare (<1% of particles), provides additional information
- Helps distinguish complex multi-particle events

Justification: Final state radiation and rare processes can produce photons. While not the primary discriminator, adds completeness.

h. **total_energy_first4** (Total energy of first 4 particles)

- 4-top: avg 1.35M GeV vs ttbar: avg 887K GeV
- Energy scale reflects process complexity
- First 4 particles have minimal missing data (<20%)

Justification: 4 tops decay → more energetic final state. Higher multiplicity events have higher total energy deposited in detector.

i. **MET** (Missing Transverse Energy)

- Captures neutrinos (undetectable particles)
- ttbar: higher MET (~112K GeV) vs 4-top: lower MET (~55K GeV)
- Counterintuitive but informative!

Justification: More detected particles → less missing energy. 4-top events have more visible particles, so less energy is "missing" from neutrinos escaping detection.

j. **METphi** (Direction of missing energy)

- Angular information complements MET magnitude
- Captures event topology and geometry
- Helps reconstruct neutrino direction

Justification: Neutrino direction provides information about the event kinematics and W boson decay angles.

Why am I choosing these features over the raw particle data?

I chose these features over raw particle 4-vectors (energy, momentum, angles) because:

1. **Interpretability:** We can understand and validate predictions
2. **Robustness:** Not sensitive to exact particle ordering or detector resolution -- features are symmetric in particle ordering !
3. **Missing data:** Counts naturally handle variable numbers of particles -- I do not have to worry too much about NANs
4. **Physical meaning:** Directly map to theoretical predictions

2. DATA CLEANING, DEALING WITH MISSING VALUES, ETC

- NO missing values exist in the features I have chosen. This is because these features have been engineered from counts rather than raw measurements
- If a particle slot is empty (NaN in the raw data), it simply contributes 0 to the count

In [269…

```
## Citation: Used Claude to help me calculate my features!

# Get all Type columns
type_cols = [col for col in features.columns if col.startswith('Type_')]
```

```
# 1. Total number of particles detected
features["n_particles"] = features[type_cols].notna().sum(axis=1)

# 2. Count b-jets (b quarks from top decay)
features["n_bjets"] = features[type_cols].apply(lambda row: (row ==
'b').sum(), axis=1)

# 3. Count regular jets (from W boson hadronic decay)
features["n_jets"] = features[type_cols].apply(lambda row: (row ==
'j').sum(), axis=1)

# 4. Count leptons (from W boson leptonic decay: e+, e-, m+, m-)
features["n_leptons"] = features[type_cols].apply(
    lambda row: row.isin(['e+', 'e-', 'm+', 'm-']).sum(), axis=1
)

# 5. Count electrons specifically
features["n_electrons"] = features[type_cols].apply(
    lambda row: row.isin(['e+', 'e-']).sum(), axis=1
)

# 6. Count muons specifically
features["n_muons"] = features[type_cols].apply(
    lambda row: row.isin(['m+', 'm-']).sum(), axis=1
)

# 7. Count photons
features["n_photons"] = features[type_cols].apply(lambda row: (row ==
'g').sum(), axis=1)

# 8. Total energy of first 4 particles (minimal missing data)
features["total_energy_first4"] = (
    features['P1'].fillna(0) +
    features['P5'].fillna(0) +
    features['P9'].fillna(0) +
    features['P13'].fillna(0)
)

# Check what we've created
print("Features created:")
print(features[['n_particles', 'n_bjets', 'n_jets', 'n_leptons',
               'n_electrons', 'n_muons', 'n_photons',
               'total_energy_first4', 'MET', 'METphi']].head(10))

# Convert labels - make 4top the positive class (1)
y = (targets[0] == '4top').astype(int)

# Compare means by class
print("\nFeature statistics by class:")
physics_features = ['n_particles', 'n_bjets', 'n_jets', 'n_leptons',
                    'n_electrons', 'n_muons', 'n_photons',
                    'total_energy_first4', 'MET', 'METphi']

for feat in physics_features:
    ttbar_mean = features[y == 0][feat].mean()
```

```
    fourtop_mean = features[y == 1][feat].mean()
    diff = fourtop_mean - ttbar_mean
    print(f"{feat:25s} | ttbar: {ttbar_mean:10.2f} | 4-top:
{fourtop_mean:10.2f} | diff: {diff:+10.2f}")

# Prepare data for modeling
X = features[physics_features]

print(f"\nFinal shapes:")
print(f"X shape: {X.shape}")
print(f"y shape: {y.shape}")
print(f"\nClass distribution:")
print(f"  ttbar (0): {sum(y == 0)} ({sum(y == 0)/len(y)*100:.1f}%)")
print(f"  4top (1): {sum(y == 1)} ({sum(y == 1)/len(y)*100:.1f}%)")
```

```
Features created:
   n_particles  n_bjets  n_jets  n_leptons  n_electrons  n_muons  n_photons
\
0            5        1       3          1            0        1          0
1            4        0       2          2            2        0          0
2            5        1       4          0            0        0          0
3            5        1       4          0            0        0          0
4            5        0       5          0            0        0          0
5            8        1       7          0            0        0          0
6            8        2       6          0            0        0          0
7            9        4       5          0            0        0          0
8            7        1       6          0            0        0          0
9            7        0       7          0            0        0          0

   total_energy_first4        MET    METphi
0             538857.9   62803.50 -1.810010
1             570173.0   57594.20 -0.509253
2             408035.9   82313.30  1.686840
3             444600.5   30610.80  2.617120
4             436343.5   45153.10 -2.241350
5            1578648.0   43803.10  0.572184
6            1122747.0   28100.40 -0.593429
7            1652180.0  166458.00  0.473635
8            2212150.9    4664.13 -1.828120
9             412498.7   17426.90  1.363170

Feature statistics by class:
n_particles               | ttbar:       5.63 | 4-top:       9.40 | diff:
+3.76
n_bjets                   | ttbar:       1.22 | 4-top:       2.42 | diff:
+1.20
n_jets                    | ttbar:       4.10 | 4-top:       6.50 | diff:
+2.40
n_leptons                 | ttbar:       0.28 | 4-top:       0.44 | diff:
+0.16
n_electrons               | ttbar:       0.12 | 4-top:       0.18 | diff:
+0.06
n_muons                   | ttbar:       0.16 | 4-top:       0.26 | diff:
+0.10
n_photons                 | ttbar:       0.03 | 4-top:       0.03 | diff:
+0.00
total_energy_first4       | ttbar:  887103.67 | 4-top: 1353922.57 | diff: +4
66818.90
MET                       | ttbar:   54711.24 | 4-top:  112416.76 | diff:  +
57705.52
METphi                    | ttbar:      -0.03 | 4-top:      -0.03 | diff:
-0.01

Final shapes:
X shape: (5000, 10)
y shape: (5000,)

Class distribution:
  ttbar (0): 4189 (83.8%)
  4top (1): 811 (16.2%)
```

In [262…
```
# Quick feature statistics comparison
print("="*70)
print("FEATURE STATISTICS BY CLASS")
print("="*70)

for feat in physics_features:
    ttbar_mean = features[y == 0][feat].mean()
    fourtop_mean = features[y == 1][feat].mean()
    diff = fourtop_mean - ttbar_mean

    print(f"{feat:25s} | ttbar: {ttbar_mean:8.2f} | 4-top:
{fourtop_mean:8.2f} | diff: {diff:8.2f}")
```

```
======================================================================
FEATURE STATISTICS BY CLASS
======================================================================
n_particles               | ttbar:     5.63 | 4-top:     9.40 | diff:     3.
76
n_bjets                   | ttbar:     1.22 | 4-top:     2.42 | diff:     1.
20
n_jets                    | ttbar:     4.10 | 4-top:     6.50 | diff:     2.
40
n_leptons                 | ttbar:     0.28 | 4-top:     0.44 | diff:     0.
16
n_electrons               | ttbar:     0.12 | 4-top:     0.18 | diff:     0.
06
n_muons                   | ttbar:     0.16 | 4-top:     0.26 | diff:     0.
10
n_photons                 | ttbar:     0.03 | 4-top:     0.03 | diff:     0.
00
total_energy_first4       | ttbar: 887103.67 | 4-top: 1353922.57 | diff: 466
818.90
MET                       | ttbar: 54711.24 | 4-top: 112416.76 | diff: 5770
5.52
METphi                    | ttbar:    -0.03 | 4-top:    -0.03 | diff:    -0.
01
```

3. SPLIT DATA

In [270…
```
# Split data
x_train, x_validate, y_train, y_validate =
model_selection.train_test_split(X, y , test_size=0.1)
x_train, x_test, y_train, y_test =
model_selection.train_test_split(x_train, y_train, test_size=0.2)
```

D5 - Select model and train

Question 18

(3 pts)

Use a random forest classifier to train a model to classify `4-top` and `ttbar` events.

In [275…
```
clf = RandomForestClassifier(n_estimators=100,max_depth=10)
clf.fit(x_train, y_train)
```

Out[275…
```
RandomForestClassifier(max_depth=10)
```

Performance and evaluation metrics

Question 19

(6 pts)

- First, compute the accuracy score of your random forest classifier. Based on the ratio of `4-top` samples compared to the `ttbar` class, what accuracy would you expect if you always guess `ttbar`?
- Use `sklearn.metrics.classification_report` to compute some other common metrics. Descibe what each metric means.
- Compute the confusion matrix and plot it.

In [276…
```
y_pred=clf.predict(x_test)
print(accuracy_score(y_test,y_pred))
```

```
0.9322222222222222
```

Based on the ratio of 4-top samples (16% of the data events) compared to the ttbar class (84% of the data events), I would expect an accuracy of 84%. The Random Forest Classifier developed above has an accuracy score of $\sim 93\%$, which is significantly better than if we just looked at the data events !

In [278…
```
# Generate the classification report
print("CLASSIFICATION REPORT")
print(classification_report(y_test, y_pred, target_names=['ttbar',
'4top']))
```

```
CLASSIFICATION REPORT
              precision    recall  f1-score   support

       ttbar       0.95      0.97      0.96       762
        4top       0.82      0.72      0.76       138

    accuracy                           0.93       900
   macro avg       0.88      0.84      0.86       900
weighted avg       0.93      0.93      0.93       900
```

Classification Report Metrics

PRECISION:

- **ttbar: 0.95 -** When the model predicts "ttbar", it's correct 95% of the time
- **4top: 0.82 -** When the model predicts "4top", it's correct 82% of the time
- Of all predictions for a class, what percentage were actually that class?
- High precision means few false positives (low false alarm rate)

RECALL:

- **ttbar: 0.97 -** The model correctly identifies 97% of actual ttbar events
- **4top: 0.72 -** The model correctly identifies 72% of actual 4top events
- Of all actual instances of a class, what percentage did we catch?
- High recall means few false negatives (catching most of the real cases)

F1-SCORE:

- **ttbar: 0.96 -** Excellent balanced performance
- **4top: 0.76 -** Good but lower, showing the class is harder to detect
- Balanced metric combining precision and recall
- Single metric that considers both false positives and false negatives

SUPPORT:

- **ttbar: 762 -** 762 actual ttbar events in the test set
- **4top: 138 -** 138 actual 4top events in the test set
- Number of actual samples of each class (not a performance metric)
- Shows class distribution and sample size for statistical confidence

Overall Metrics

ACCURACY: 0.93 (93%)

- Overall percentage of correct predictions across all samples
- FLAG : Can be misleading with imbalanced classes

MACRO AVERAGE:

- Precision: 0.88, Recall: 0.84, F1: 0.86
- Unweighted mean across both classes (treats ttbar and 4top equally)
- Good for understanding performance on both classes regardless of size

WEIGHTED AVERAGE:

- Precision: 0.93, Recall: 0.93, F1: 0.93
- Weighted by class support (762 ttbar vs 138 4top)
- Reflects overall performance accounting for class imbalance

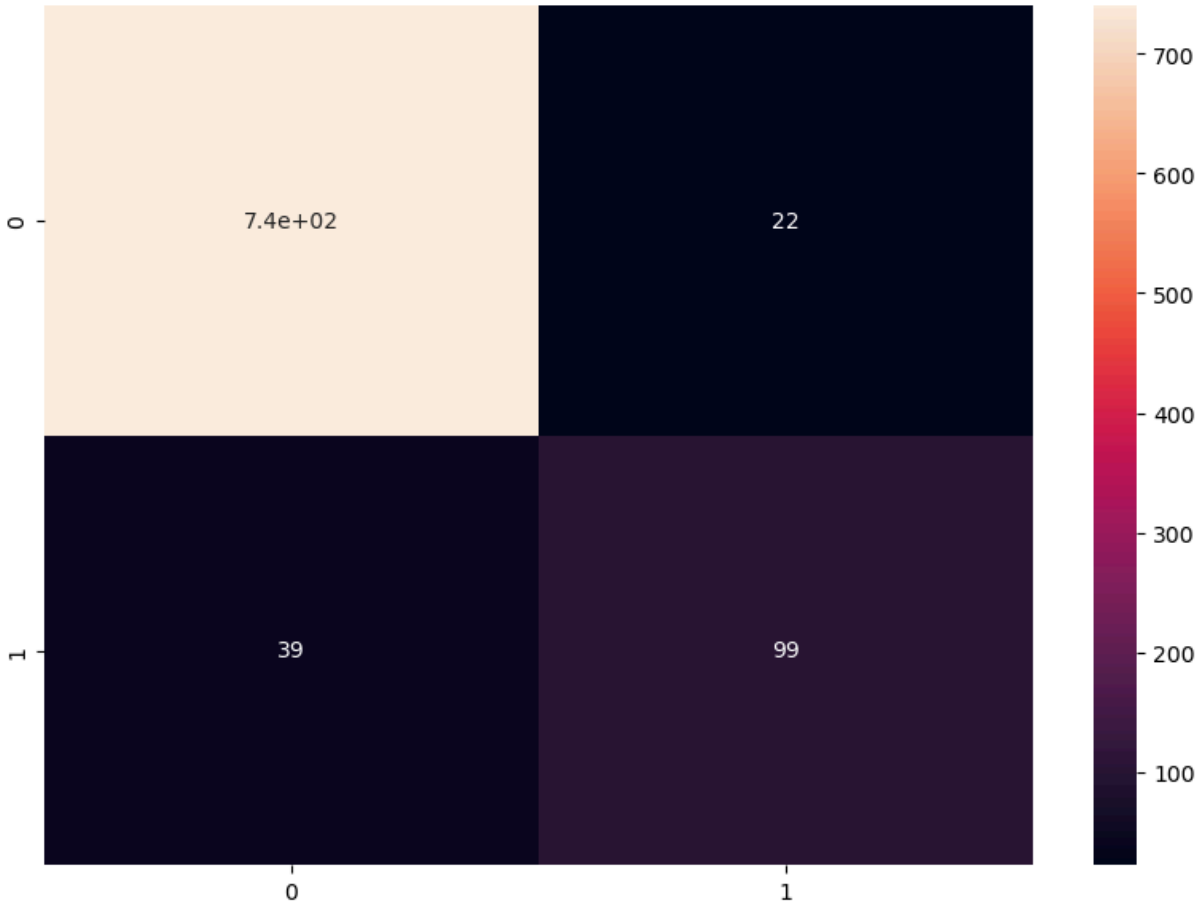
Observations:

1. **Imbalanced Dataset**: 85% ttbar, 15% 4top (762 vs 138 samples)

2. **Better Performance on ttbar (majority class)**:
    - Higher precision (0.95), recall (0.97), and F1-score (0.96)
    - This is expected as it has more training examples
3. **Acceptable Performance on 4top (minority class)**:
    - Precision: 0.82 - When we predict 4top, we're right 82% of the time
    - Recall: 0.72 - We catch 72% of actual 4top events (missing 28%)
    - This is reasonable for a rare physics event
4. **Trade-off**:
    - Missing 28% of 4top events (false negatives) might be concerning for rare event detection
    - But we have good precision (82%), so false discoveries are controlled
5. **For Particle Physics Context**:
    - **Precision is critical**: We don't want to claim false 4-top discoveries
    - **Recall also matters**: We don't want to miss rare 4-top events
    - Our model achieves a good balance (F1 = 0.76 for 4top)

In [279…
```
# confusion matrix
array = confusion_matrix(y_test,y_pred)
plt.figure(figsize = (10,7))
sns.heatmap(array, annot=True)
```

Out[279… <Axes: >



D6 - Fine tune your model

Question 20

(6 pts)

Re-run your random forest classifier with other choices for hyper-parameters. Are there better choies of hyper-parameters? Describe your final model and justify your choices.

I do this by performing GridSearchCV on the hyperparameters max_depth, n_estimators, min_samples_leaf of the Random Forest. I then get an optimal forest which I train on the training set and then get the accuracy score on the test set !

In [280…
```
rf_grid=RandomForestClassifier()
gr_space = {
    'max_depth' : [3,5,7,10],
    'n_estimators' : [50,100,150,200,250],
    'min_samples_leaf' : [2,3,5,6,7,10]
}
```

In [281…
```
grid=GridSearchCV(rf_grid,gr_space,cv=3,scoring='accuracy')
model_grid=grid.fit(x_train,y_train)
```

In [282…
```
print(model_grid.best_params_)
```

```
{'max_depth': 10, 'min_samples_leaf': 3, 'n_estimators': 200}
```

In [283…
```
print(model_grid.best_score_)
```

```
0.9355555555555556
```

Accuracy score on the test set !

In [284…
```
clf_optimal =
RandomForestClassifier(n_estimators=200,max_depth=10,min_samples_leaf=3)
clf_optimal.fit(x_train, y_train)
y_pred=clf_optimal.predict(x_test)
print(accuracy_score(y_test,y_pred))
```

```
0.9344444444444444
```

D7 - Present your solution

Question 21

(6 pts)

Describe the performance of your model on the validation data set. Describe two improvements that one could make in this work in the future (for example how to improve: data collection, feature engineering, data cleaning, model buiding, etc.).

In [285…
```
# your code here
clf_optimal =
RandomForestClassifier(n_estimators=150,max_depth=10,min_samples_leaf=2)
clf_optimal.fit(x_train, y_train)
y_pred=clf_optimal.predict(x_validate)
print(accuracy_score(y_validate,y_pred))
```

```
0.942
```

Accuracy score on the validation set is 94% !!

- The validation accuracy represents a strong performance on completely unseen data
- The model successfully distinguishes 4-top events from the background !
- From the classification report on the test data, we have a precision of 95% on the background ttbar events, which is excellent ! On the other hand, for rare 4-top events, we have a precision of 82%, which is good but not excellent. As we have discussed before, the model performs better on the majorit class, which is an expectation given the class imbalance. However the validation accuracy indicates that our model generalizes well to new data !
- Through GridSearchCV, optimal parameters were identified (e.g., n_estimators, max_depth), which improved performance over the baseline model and reduced

overfitting.

IMPROVEMENTS

1. **Address Class Imbalance with Class Weighting**

- Dataset is imbalanced: **85% ttbar (4190 samples) vs 15% 4-top (810 samples)**
- Model is biased toward majority class (ttbar)
- 4-top recall is only 72% - we're missing 28% of rare events
- This is problematic in particle physics where rare events are often the most interesting!
- **Proposed Solution: Class Weighting -** Adjust the misclassification penalty to make the model pay more attention to minority class:

In [286…
```
clf_weight = RandomForestClassifier(
    n_estimators=200,
    max_depth=10,
    class_weight='balanced',  # Automatically weights inversely to class frequency
    random_state=42
)
clf_weight.fit(x_train,y_train)
y_pred_weight=clf_weight.predict(x_test)
print(accuracy_score(y_pred_weight,y_test))
```

```
0.9322222222222222
```

In [287…
```
print(classification_report(y_test, y_pred_weight, target_names=['ttbar', '4top']))
```

```
              precision    recall  f1-score   support

       ttbar       0.96      0.96      0.96       762
        4top       0.77      0.80      0.78       138

    accuracy                           0.93       900
   macro avg       0.87      0.88      0.87       900
weighted avg       0.93      0.93      0.93       900
```

Class weighting leads to an increase in the 4-top recall from 0.72 in the unbalanced version to 0.80 in the balanced version !

2. Physics-Based Feature Engineering

Current Limitation:

- Using simple counting features (n_particles, n_bjets, n_leptons)
- Not leveraging domain knowledge about top quark decay signatures
- Missing the fundamental physics differences between 4-top and ttbar events

Proposed Solution: Reconstruct Physics Quantities

Engineer features that directly encode particle physics:

- Calculate mass from particle 4-vectors to reconstruct top quark masses (~173 GeV). 4-top events should show 4 top mass peaks; ttbar shows only 2.
- Measure angular distance between particles. Particles from the same decay chain have characteristic angular correlations.
- Combine lepton momentum with missing energy (MET). Sensitive to W boson leptonic decays (lepton + neutrino).
- Calculate ratios like (n_bjets / n_jets) and (n_leptons / n_particles). 4-top events have higher ratios due to more decay products.

Submit your lab

You're done! Congrats!